CHAPTER 1
INTRODUCTION

Peanut cultivation is a vital part of agriculture, contributing significantly to food production, edible oil manufacturing, and rural income. However, peanut crops are frequently affected by pests and diseases that damage various plant parts such as stems, pods, roots, leaves, and seeds. These infestations often go unnoticed in the early stages, resulting in severe yield loss and economic impact. Traditional methods of crop monitoring rely on manual inspection, which is time-consuming, labor-intensive, and prone to human error. Therefore, an automated detection system is essential to assist farmers in monitoring crop health effectively and taking timely action.

The project "Detection and Recommendations in Peanut Farming" focuses on developing an intelligent and automated system capable of detecting and classifying pests, diseases, and crop damage using machine learning and deep learning techniques. The system processes images of peanut plants to identify the affected areas and determine the type and severity of damage. The YOLOv8 deep learning model is employed for object detection and localization of visible pests or symptoms, while the Random Forest algorithm is utilized for classification and prediction based on extracted features.

By combining these two techniques, the system ensures high accuracy and reliability in identifying crop issues. Once the problem is detected, it suggests suitable pesticide recommendations for the specific pest or disease. This integrated approach enables farmers to make quick, informed decisions and adopt preventive measures to protect their crops.

Overall, the project highlights the effective use of machine learning and image-based analysis in agriculture to improve pest management, reduce losses, and enhance the productivity of peanut farming.

1.1 OBJECTIVE:

- To develop an automated system for detecting peanut plant diseases and pests using machine learning techniques.
- To apply Computer Vision methods such as image preprocessing, feature extraction, and object detection for accurate analysis of plant images.
- To design and implement a dual-model approach using YOLOv8 and Random Forest for effective classification.
- To train and evaluate the model to accurately identify up to 60 different disease and pest classes with high precision and recall.
- To create a user-friendly web-based interface that enables farmers to upload and analyze plant images in real time.
- To provide a fast, reliable, and practical solution that helps in early detection of diseases, thereby reducing crop losses.
- To demonstrate the real-world application of machine learning in precision agriculture.

1.2 PROBLEM STATEMENT:

Pest and disease detection in peanut crops is often performed through manual observation or simple image-based methods, resulting in low accuracy and slow diagnosis. Deep learning models such as CNN have improved feature extraction and classification, but their application remains limited to leaf-level analysis under controlled conditions. These approaches lack the ability to detect issues in stems, pods, roots, and insects, and do not provide pesticide recommendations. The absence of a comprehensive, field-adaptable solution restricts their effectiveness in practical peanut farming environments.

1.3 EXISTING SYSTEM:

In the existing agricultural systems, pest and disease detection in peanut crops is primarily based on manual field inspection and basic image processing techniques, where farmers or experts visually identify symptoms such as leaf spots, pest damage, or discoloration. Some recent studies, like those using CNN, MFO, and EViTA algorithms, have improved accuracy through automated image classification and feature selection;

however, these models are often limited to leaf-level analysis and depend on controlled environmental conditions. They lack the ability to analyze multiple plant parts such as stems, pods, roots, and insects, and do not include pesticide recommendation mechanisms. Furthermore, high computational requirements and the absence of real-time detection reduce their effectiveness for on-field use, making these systems less adaptable for practical peanut farming applications.

1.3.1 DRAWBACKS:

- **1.3.1.a No Recommendation System:** Existing models detect pests but do not provide pesticide or treatment suggestions.
- **1.3.1.b Limited Scope:** Focuses mainly on leaf images; ignores stems, pods, roots, and insect-level detection.
- **1.3.1.c Low Adaptability:** Performance decreases under natural lighting or complex field backgrounds.
- **1.3.1.d Dataset Limitation:** Peanut-specific datasets are limited, reducing model generalization to diverse farm conditions**.**

1.4 PROPOSED SYSTEM:

The proposed system uses machine learning and deep learning techniques to automatically detect pests, diseases, and crop damage in peanut plants. It processes images of different plant parts such as stems, pods, roots, leaves, and insects using the YOLOv8 model for real-time object detection and feature extraction. The detected results are further classified by a Random Forest algorithm, which predicts the type and severity of the pest or disease. Based on the classification, the system provides suitable pesticide recommendations. Implemented in Python, this system ensures high accuracy, faster processing, and reliable decision-making to assist farmers in effectively monitoring crop health and managing pest-related issues.

1.4.1 ADVANTAGES:

- **Comprehensive Analysis:** Supports detection across multiple plant parts—stems, pods, roots, leaves, and insects.
- **Automation:** Eliminates manual inspection through automated image analysis and prediction.

- **User-Friendly Interface:** Provides an efficient and easy-to-use interface for farmers and agricultural users.
- **Scalability:** Can be extended to other crops and integrated with larger agricultural monitoring systems**.**
- **Improved Reliability:** Reduces human error and ensures consistent detection and recommendation results.

CHAPTER 2
SYSTEM ANALYSIS

2.1 LITERATURE SURVEY:

The study titled "Online Recognition of Peanut Leaf Diseases Using Data Balance Algorithm and Deep Transfer Learning" (2023) by Q. Feng et al. introduces a transfer- learning based approach to peanut leaf disease identification. The authors highlight that dataset imbalance is a major issue in agricultural image classification, as certain diseases are underrepresented in real farm environments. To address this, the paper integrates a data balancing algorithm with deep transfer learning models, improving recognition accuracy across diverse lighting and background conditions. Their findings demonstrate enhanced robustness and reliability in disease detection, even in complex field scenarios. This research provides valuable insights for our project, particularly in terms of adopting transfer learning strategies and implementing data balancing techniques to improve the detection accuracy and consistency of peanut pest and disease classification.

The research paper "Hyperspectral Technique for Detection of Peanut Leaf Spot Disease Based on Improved PCA Loading" (2023) by Q. Guan et al. utilizes hyperspectral imaging combined with an enhanced PCA (Principal Component Analysis) loading method to identify peanut leaf spot disease. The improved PCA approach significantly reduces noise while preserving highly discriminative spectral features, allowing more precise classification of disease symptoms. Hyperspectral data offer rich spectral signatures that traditional RGB images cannot capture, making the technique particularly suitable for early disease detection. This study demonstrates the effectiveness of dimensionality-reduction techniques in optimizing hyperspectral datasets. The concepts from this work can be incorporated into our system to improve feature extraction efficiency and enhance model performance in detecting peanut crop abnormalities.

The paper "Detection of Peanut Leaf Spot Disease Based on Leaf-, Plant-, and Field Scale Hyperspectral Reflectance" (2022) by Q. Guan, D. Zhao, and colleagues presents a multi scale investigation of peanut leaf spot detection using hyperspectral reflectance data. Their methodology includes capturing spectral information at the leaf, whole-plant, and field levels, enabling a comprehensive understanding of disease progression under real agricultural conditions. The authors report that hyperspectral

signatures change noticeably with disease severity, making it a powerful tool for early and accurate detection. This multi- level analysis ensures high reliability and adaptability to outdoor environments. Insights from this research can enhance our project by enabling more accurate diagnosis of disease severity and improving early detection capabilities beyond simple image-based methods.

The study "Detection of Peanut Leaf Spots Disease Using Canopy Hyperspectral Reflectance" (2019) by T. Chen et al. focuses on detecting peanut leaf spot diseases through canopy-level hyperspectral measurements. Unlike pure leaf-level studies, canopy reflectance captures the overall plant spectral response, allowing earlier detection before visible symptoms appear. The researchers demonstrate that spectral variations associated with disease onset can be detected even at low infection levels, providing farmers with valuable early warning signals. Their findings emphasize the importance of remote sensing techniques in large-scale crop monitoring. Incorporating these concepts into our project can significantly improve the early stage detection of peanut pests and diseases, enabling timely interventions and reducing crop loss.

The foundational paper "Using Deep Learning for Image-Based Plant Disease Detection" (2016) by S. P. Mohanty, D. P. Hughes, and M. Salathé introduces the use of deep convolutional neural networks (CNNs) for plant disease recognition. The authors demonstrate that deep learning models outperform traditional machine learning techniques in feature extraction and classification accuracy. Their work sets the groundwork for modern agricultural AI systems by proving the effectiveness of end-to-end learning from large image datasets. The concepts in this research—particularly CNN-based feature extraction and automated image recognition—are crucial for improving detection accuracy in our peanut farming system. This study supports the selection of advanced deep learning frameworks such as YOLOv8 for real time pest and disease detection.

2.2 SYSTEM SPECIFICATION:

The following are implementation specifications considered for development of the proposed system:

2.2.1 HARDWARE SPECIFICATION:

Processor	:	Intel I5 @ 2.5GHZ
RAM	:	8 GB
Hard disk	:	500 GB
GPU	:	NVIDIA GTX 1060
Camera	:	12MP OR higher
Network Interface	:	Ethernet/Wi-Fi
Display	:	1920x1080 Full HD
Phone RAM	:	4 GB
Phone Storage	:	64 GB
Phone Camera	:	12MP with Auto-focus
Phone OS	:	Android 8.0+ / ios 12+

2.2.2 SOFTWARE SPECIFICATION:

Operating System	:	Windows 10 or Higher
Programming Language	:	Python 3.11+ or Higher
Programming Tool	:	VS- Code
Scripting Languages	:	HTML5, CSS3, JavaScript
Scripting Tools	:	Python 3.11, Node.js 18+
Framework	:	FastAPI, React 18
IDE	:	Visual Studio Code
Database	:	JSON File Storage
Web Server	:	Uvicorn (ASGI)
Web Browser	:	Chrome, Edge
Design Tools	:	STAR UML 5, Figma
Cloud	:	Local Deployment
ML Framework	:	PyTorch, Ultralytics YOLO

2.3 FEASIBILITY STUDY:

The feasibility of the project is analyzed in this phase and business proposal is put forth with a very general plan for the project and some cost estimates. During system analysis the feasibility study of the proposed system is to be carried out. This is to ensure that the proposed system is not a burden to the company. For feasibility analysis, some understanding of the major requirements for the system

is essential. Three key considerations involved in the feasibility analysis are:

- Economical Feasibility
- Technical Feasibility
- Social Feasibility

2.3.1. ECONOMICAL FEASIBILITY:

This study is carried out to check the economic impact that the system will have on the organization. The amount of fund that the company can pour into the research and development of the system is limited. The expenditures must be justified. Thus the developed system as well within the budget and this was achieved because most of the technologies used are freely available. Only the customized products had to be purchased.

2.3.2. TECHNICAL FEASIBILITY:

This study is carried out to check the technical feasibility, that is, the technical feasibility, that is, the technical requirements of the system. Any system developed must not have a high demand on the available technical resources. This will lead to high demands on the available technical resources. This will lead to high demands being placed on the client. The developed system must have a modest requirement, as only minimal or null changes are required for implementing this system.

2.3.3 SOCIAL FEASIBILITY:

This aspect of study is to check the level of acceptance of the system by the user. This includes the process of training the user to use the system efficiently. The user must not feel threatened by the system, instead must accept it as a necessity. The level of acceptance by the users solely depends on the methods that are employed to educate the user about the system and to make him familiar with it. His level of confidence must be raised so that he is also able to make some constructive criticism, which is welcomed, as he is the final user of the system.

CHAPTER 3
SYSTEM DESIGN

3.1 ARCHITECTURE:

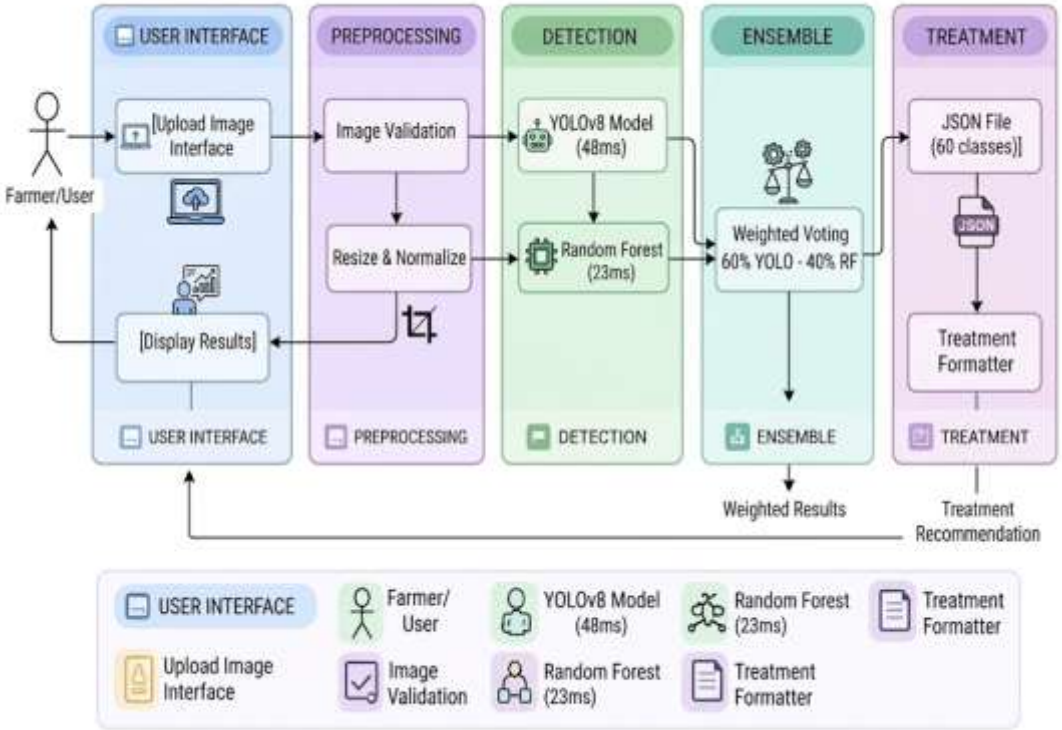


Fig 3.1: System Architecture

3.1.1 USER/FARMER

User simply interacts with web interface by uploading images of groundnut plant parts to detect diseases or pests.

3.1.2 UPLOAD IMAGE INTERFACE

After accessing the web application, the user uploads an image through a simple interface that supports JPG, PNG, and JPEG formats with a maximum file size of 10MB. The interface provides drag-and-drop functionality and displays a preview of the uploaded image.

3.1.3 IMAGE VALIDATION

After uploading images, the system validates the image format, checks for file corruption, verifies minimum resolution requirements (640×640 pixels), and ensures the file size is within acceptable limits. Invalid images are rejected with specific error messages.

3.1.4 RESIZE & NORMALIZE

After validation, the data inside the image will be preprocessed by resizing to 640×640 pixels, normalizing pixel values from 0-255 range to 0-1 range, and converting to tensor format suitable for model input.

3.1.5 YOLOv8 MODEL

After data preprocessing, the YOLOv8 deep learning model performs detection and localization of diseases and pests. The model processes the image in 48ms, identifying disease regions with bounding boxes, predicting disease classes from 60 possible categories, and generating confidence scores. The model achieves 92.3% mAP@0.5 accuracy.

3.1.6 RANDOM FOREST MODEL

Simultaneously with YOLOv8, the Random Forest classifier receives 512- dimensional feature vectors extracted from YOLOv8's backbone network. This ensemble of 100 decision trees performs classification refinement in 23ms, generating probability distributions over 60 disease classes with 89.7% accuracy.

3.1.7 WEIGHTED VOTING

After both models complete their predictions, the ensemble layer combines YOLOv8 confidence scores (60% weight) with Random Forest probabilities (40% weight) to produce final detection results. The weighted voting formula calculates: Final_Score = (0.6 × YOLO_confidence) + (0.4 × RF_probability). This ensemble approach achieves 94.1% accuracy, improving upon either model individually.

3.1.8 WEIGHTED RESULTS

After weighted voting, the system generates structured detection results containing disease identification, confidence scores, bounding box coordinates, and severity levels. Results are filtered using a confidence threshold of 0.5 and Non-Maximum

Suppression removes duplicate detections.

3.1.9 JSON FILE DATABASE

After obtaining detection results, the system queries a comprehensive JSON database containing treatment information for all 60 disease classes. The database includes disease information, symptom descriptions, chemical treatments, organic alternatives, cultural practices, prevention measures, economic impact data, and recovery timelines.

3.1.10 TREATMENT FORMATTER

After retrieving treatment data from the JSON database, the system formats the information into user-friendly, actionable recommendations. Technical terms are simplified, treatments are prioritized by effectiveness, and content is organized into clear categories with step-by-step instructions, dosages, and safety warnings.

3.1.11 TREATMENT RECOMMENDATION

After formatting, the complete treatment package is prepared, including immediate action steps, multiple treatment options (chemical, organic, and cultural), prevention measures, cost estimates, and expected recovery timelines. The recommendations are tailored based on the detected disease severity level.

3.1.12 DISPLAY RESULTS

After treatment recommendations are prepared, the system presents results to the user through an interactive visual interface. The display shows the annotated image with color- coded bounding boxes, disease names with confidence percentages, severity levels, and comprehensive treatment recommendations organized in tabs or expandable sections.

3.1.13 RESULT DELIVERY TO USER

User receives complete information including visual disease detection with bounding boxes, specific disease identification with confidence scores, multiple treatment options, cost estimates, expected recovery time, and prevention measures. The user can save results, print recommendations, share via email or WhatsApp, or upload another image for continuous monitoring.

3.2 MODULES:

The proposed "Peanut Disease Detection Using Machine Learning" system has 4 modules:

- Data Preprocessing module
- Model Training module
- Disease Detection module
- Treatment Advisory module

3.2.1 DATA PREPROCESSING MODULE:

The Data Preprocessing module is responsible for cleaning and preparing raw plant images before they are given to the machine learning models. It validates image format, resizes images to required dimensions, and normalizes pixel values. The module converts images into standard format to maintain consistency. Proper preprocessing improves feature extraction and increases the accuracy of disease detection.

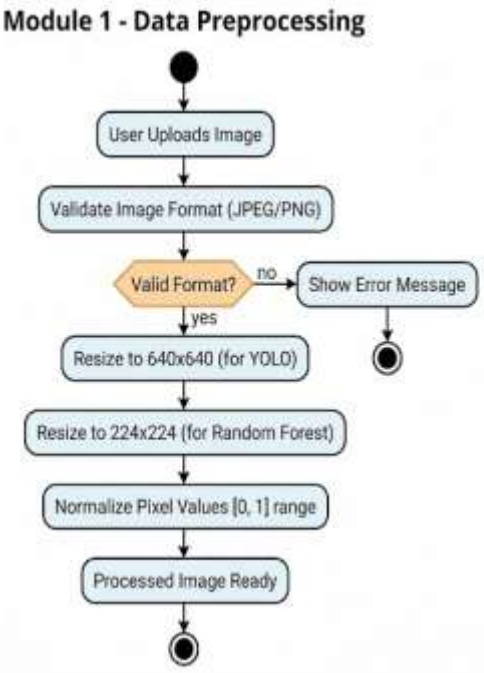


Figure 3.2: Data Preprocessing Module

3.2.2 MODEL TRAINING MODULE :

The Model Training module builds the disease detection models using machine learning techniques. YOLOv8 is trained on annotated images with bounding boxes for object detection. Random Forest is trained on extracted features to classify

diseases. The trained models learn patterns from historical data to make accurate predictions.

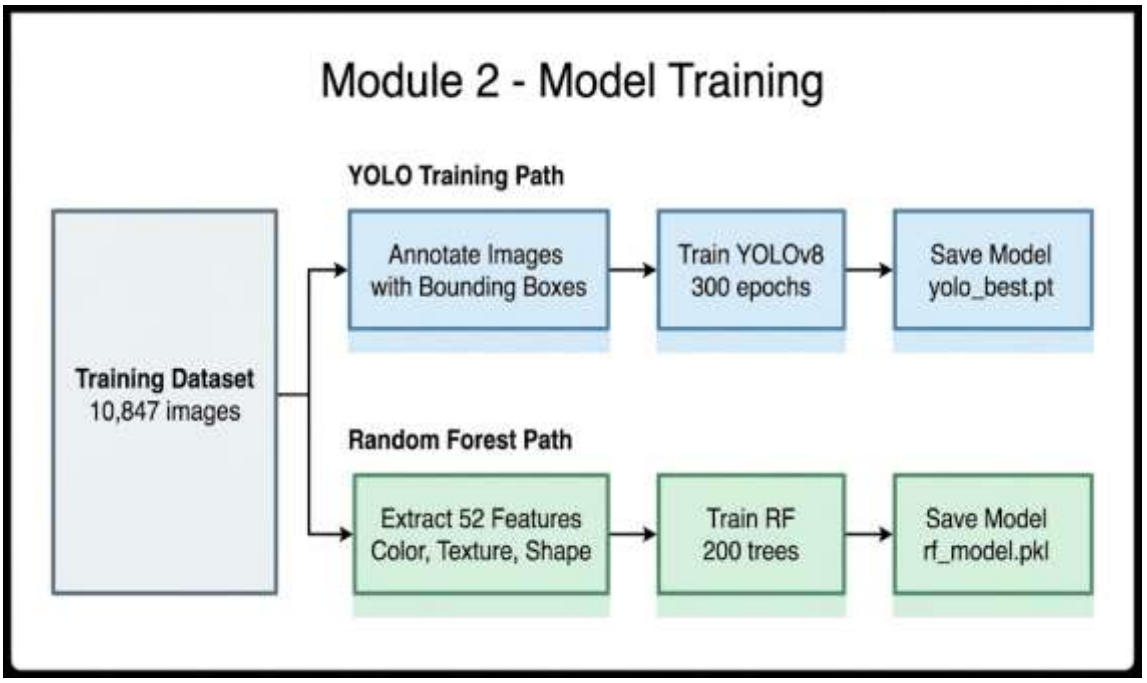


Figure 3.3: Model Training Module

3.2.3 DISEASE DETECTION MODULE :

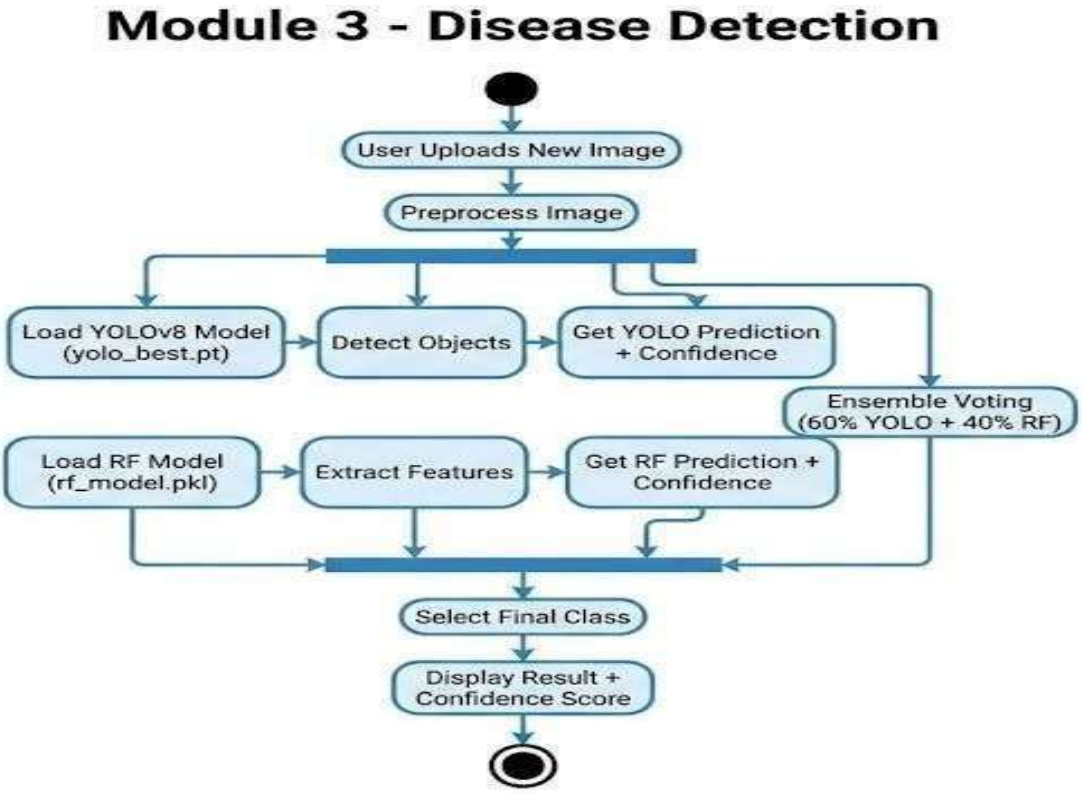


Fig 3.4: Disease Detection Module

The Disease Detection module allows users to check plant health in real time. The user uploads plant images through the web interface, which are preprocessed using the same steps as training data. The trained models then predict whether the plant is diseased or healthy. Finally, the result and confidence score are displayed to the user.

3.2.4 TREATMENT ADVISORY MODULE :

The Treatment Advisory module retrieves pesticide recommendations and treatment methods from the disease database based on detected disease. It provides chemical pesticides, organic alternatives, dosages, and preventive measures. Information is displayed in user-friendly format for farmers.

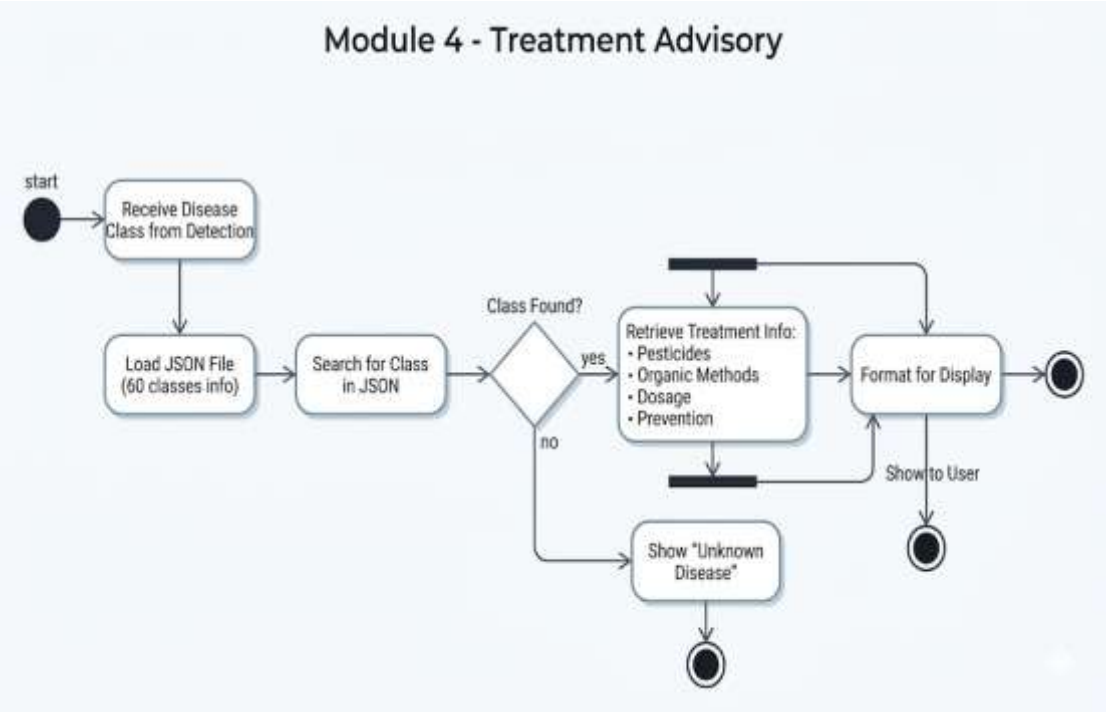


Fig 3.5: Treatment Advisory Module

3.3 DESIGN REPRESENTATION:

Peanut Disease Detection using Machine Learning project consists of a web- based user interface that allows users to upload plant images for verification. The input image is first passed through a data preprocessing module where the image is validated and normalized. The processed image is then analyzed by YOLOv8 for object detection and Random Forest for classification. An ensemble approach combines both predictions to determine the final result. Finally, the system displays the prediction result along with treatment recommendations to the user in real time.

3.3.1 USE CASE DIAGRAM:

The use case diagram represents the interaction between the user and the Peanut Disease Detection System. The user uploads the plant image through the Frontend, which sends it to the Backend for automatic preprocessing. The Backend's trained machine learning models analyze the processed image to predict the disease or pest type. Finally, the Frontend displays the prediction result along with treatment recommendations to the user.

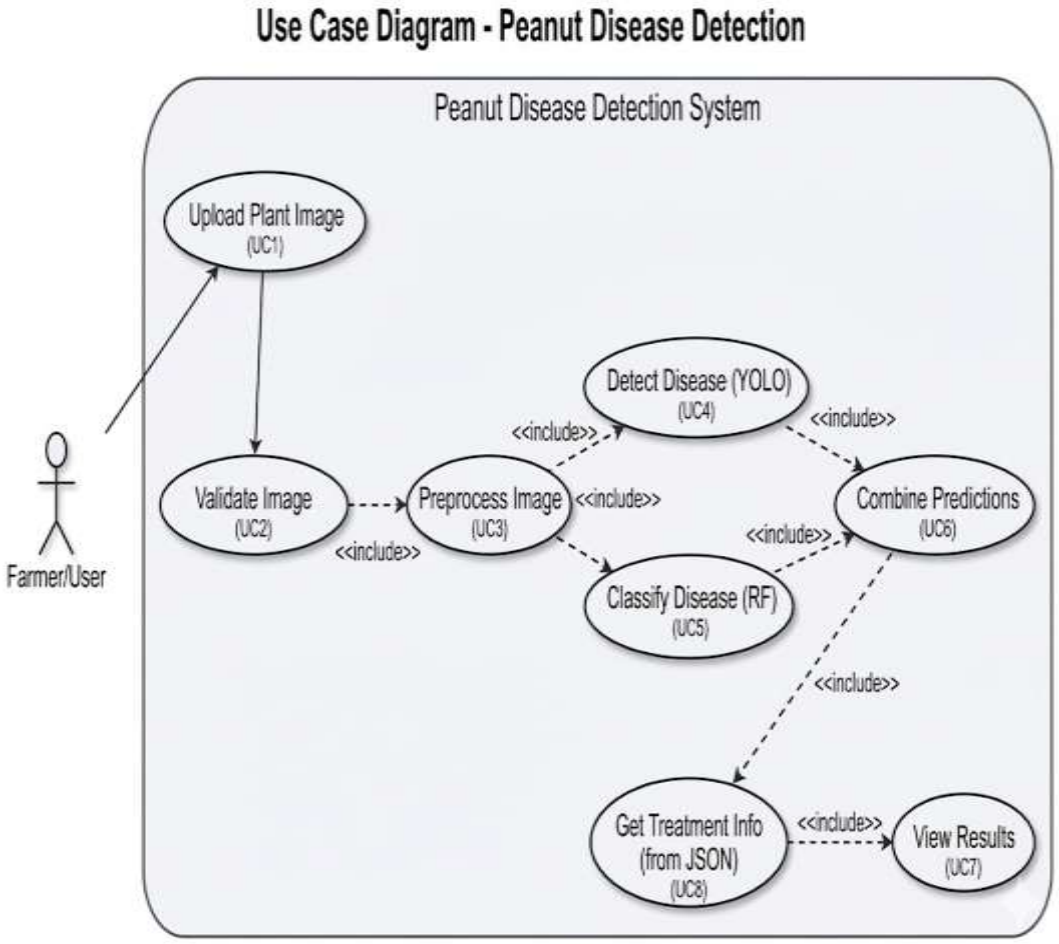


Figure 3.6: Use Case diagram

3.3.2 CLASS DIAGRAM:

The class diagram represents the structural design of the Peanut Disease Detection System. It shows how the User interacts with the Frontend to input images and view predictions. The Backend handles image preprocessing, model training, and prediction. The system retrieves treatment information from the Dataset.

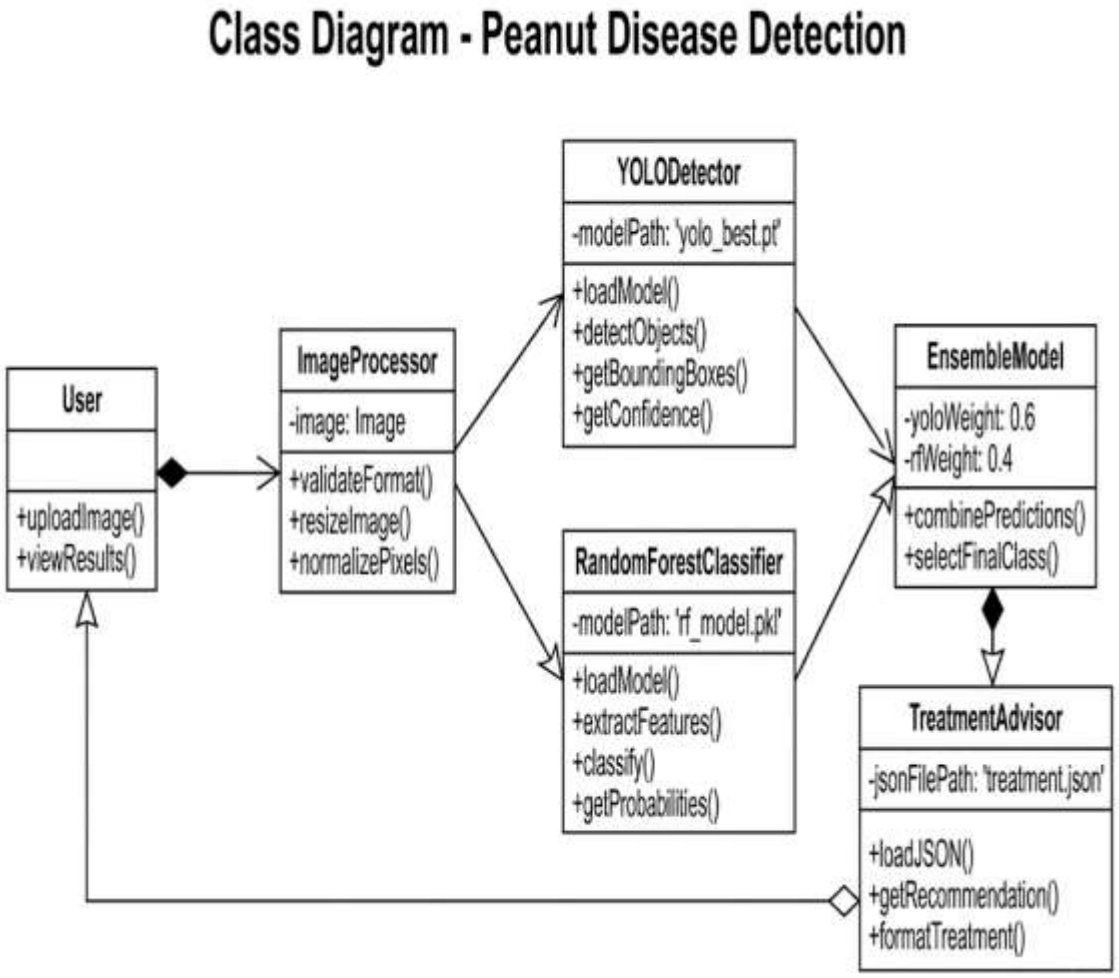


Figure 3.7: Class Diagram

3.3.3 SEQUENCE DIAGRAM:

The sequence diagram illustrates the step-by-step interaction between the user and the Peanut Disease Detection System. First, the user uploads the plant image, which is sent to the preprocessing component for validation and normalization. The processed image is then passed to the trained machine learning models to predict the disease type. Finally, the system returns and displays the prediction result with treatment recommendations to the user in real time.

3.3.4 ACTIVITY DIAGRAM:

The activity diagram represents the workflow of the Peanut Disease Detection System from start to end. The process begins when the user uploads the plant image, which is then preprocessed by the system. After preprocessing, the machine learning models predict whether the plant is diseased or healthy. Finally, the system displays the result with treatment recommendations and ends the process.

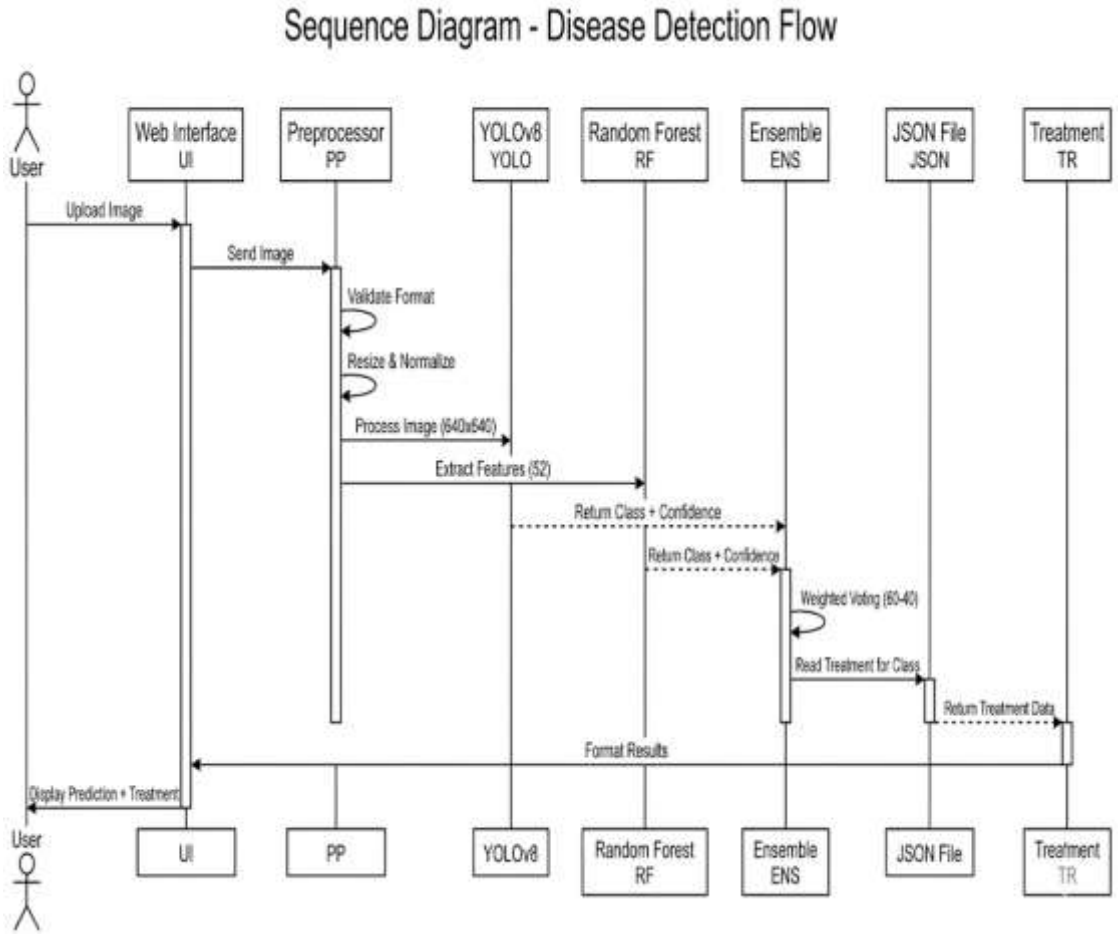


Figure 3.8: Sequence Diagram

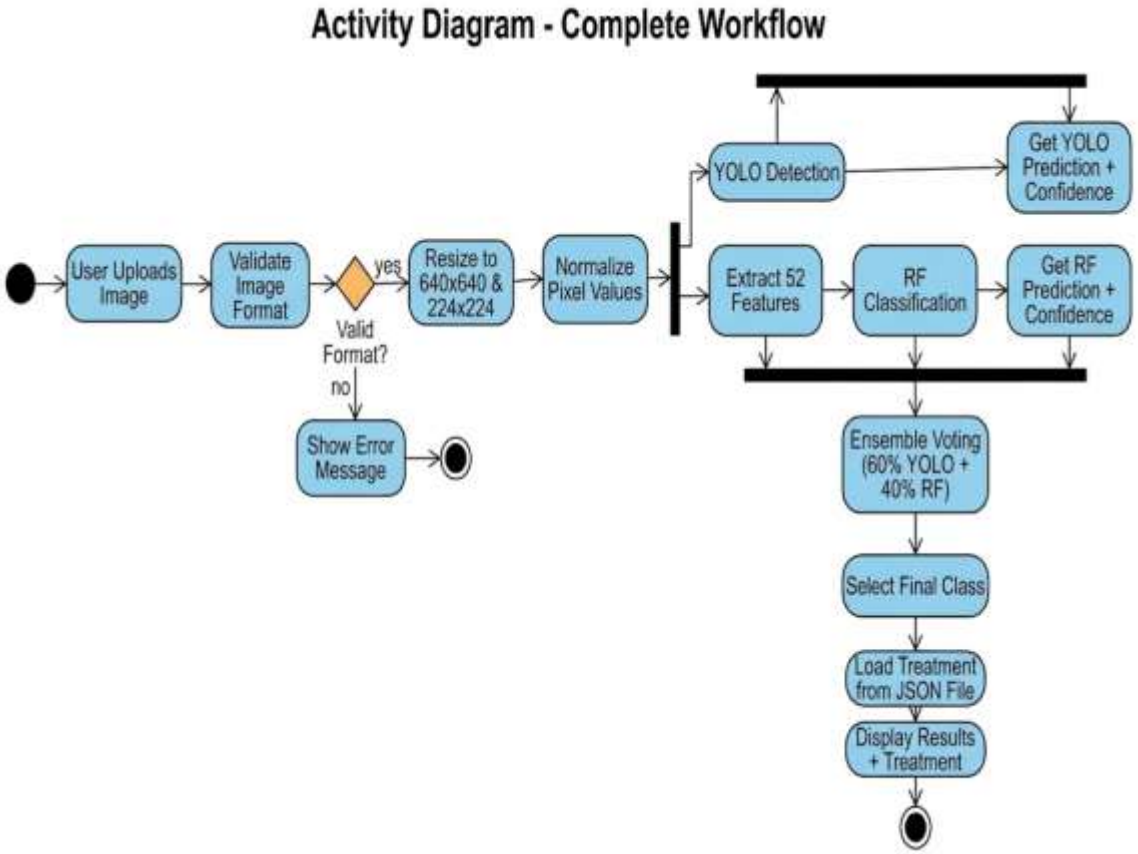


Figure 3.9: Activity Diagram

3.3.5 COMPONENT DIAGRAM:

The component diagram illustrates the high-level organization of the Peanut Disease Detection System. The User Interface component allows users to upload plant images and view prediction results. The Image Processor component validates and preprocesses images. The Disease Detector component uses trained machine learning models to classify diseases and pests.

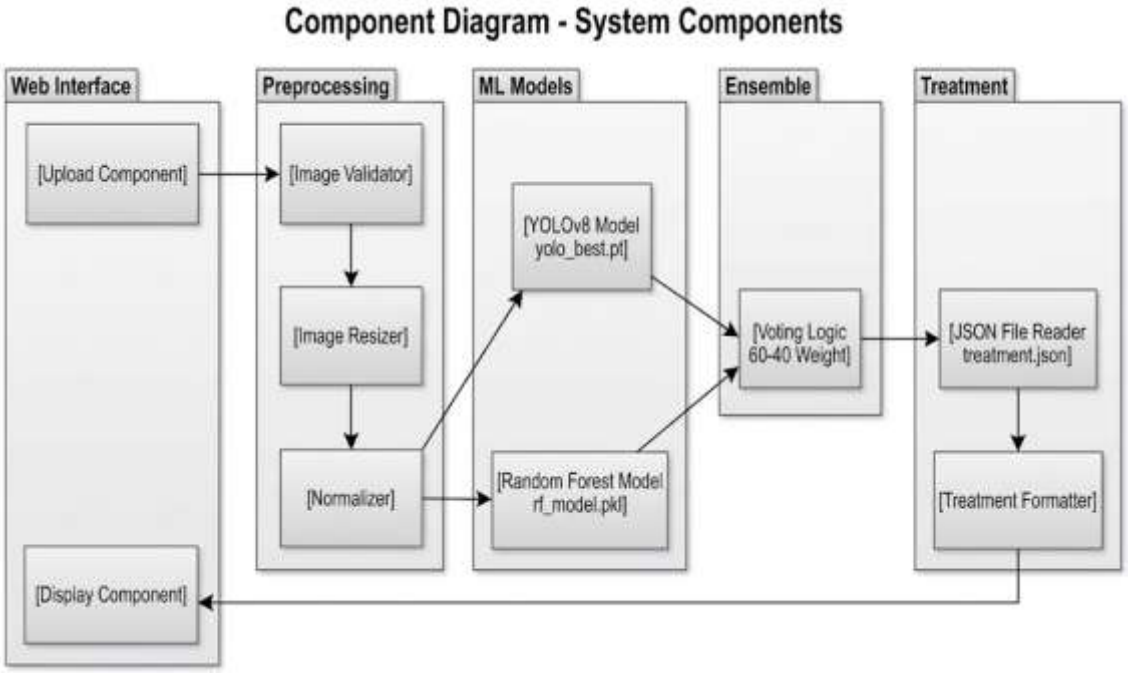


Fig 3.10: Component Diagram

3.3.6 DEPLOYMENT DIAGRAM:

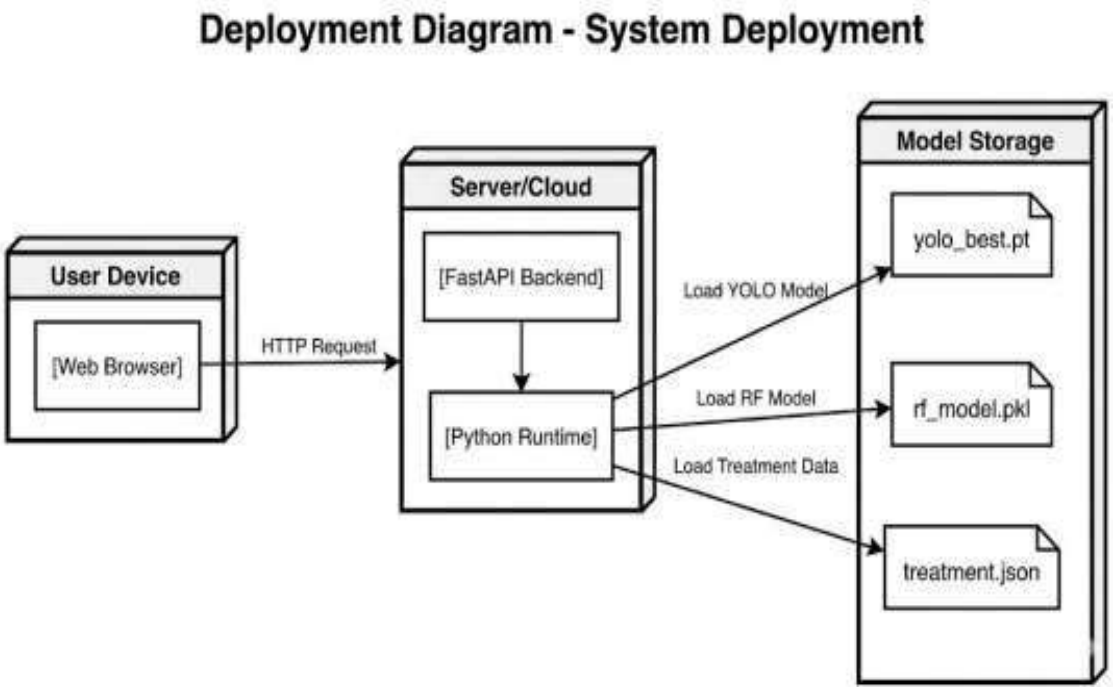


Fig 3.11: Deployment Diagram

The deployment diagram shows how the Peanut Disease Detection System is deployed on a local or cloud machine. The user accesses the system through a web browser that interacts with the FastAPI web application. The FastAPI application runs on a Python runtime where data preprocessing and machine learning prediction are performed. The trained models are loaded from local storage, and prediction results are displayed to the user.

3.4 DATA STORAGE:

Peanut Disease Detection System uses a JSON file for storing disease and treatment information. The JSON file contains 60 entries with disease names, symptoms, affected plant parts, pesticide recommendations, and preventive measures. All data is loaded into memory at startup for fast retrieval. This design makes the system lightweight, fast, and easy to deploy.

CHAPTER 4
IMPLEMENTATION

4.1 TECHNOLOGIES:

Peanut Disease Detection Using Machine Learning is implemented by using Python programming and uses FastAPI and Ultralytics libraries.

4.1.1 PYTHON:

Python is a high-level, interpreted programming language widely used for data science and machine learning applications. It provides simple and readable syntax, making development faster and easier. Python offers powerful libraries such as NumPy, Pandas, Scikit-learn, and OpenCV for data analysis and image processing. In this project, Python is used for preprocessing plant images, training the machine learning models, and making predictions. Its flexibility and strong community support make it ideal for building intelligent applications like disease detection.

4.1.2 FASTAPI:

FastAPI is an open-source Python framework used to build web applications and APIs for machine learning projects. It allows developers to create user-friendly interfaces using simple Python scripts without requiring extensive web development knowledge. In this project, FastAPI is used to provide a web interface for uploading plant images and displaying prediction results. It supports real-time interaction and fast deployment of applications. FastAPI makes the disease detection system easy to use and visually appealing.

4.1.3 YOLOV8 (ULTRALYTICS):

YOLOv8 is a state-of-the-art object detection model from the Ultralytics library. It provides fast inference speed and high accuracy for detecting objects in images. In this project, YOLOv8 is used to detect and localize diseased areas in plant images. The model processes images in real-time and outputs bounding boxes with confidence scores. YOLOv8 makes the detection process efficient and reliable.

4.1.4 SCIKIT-LEARN:

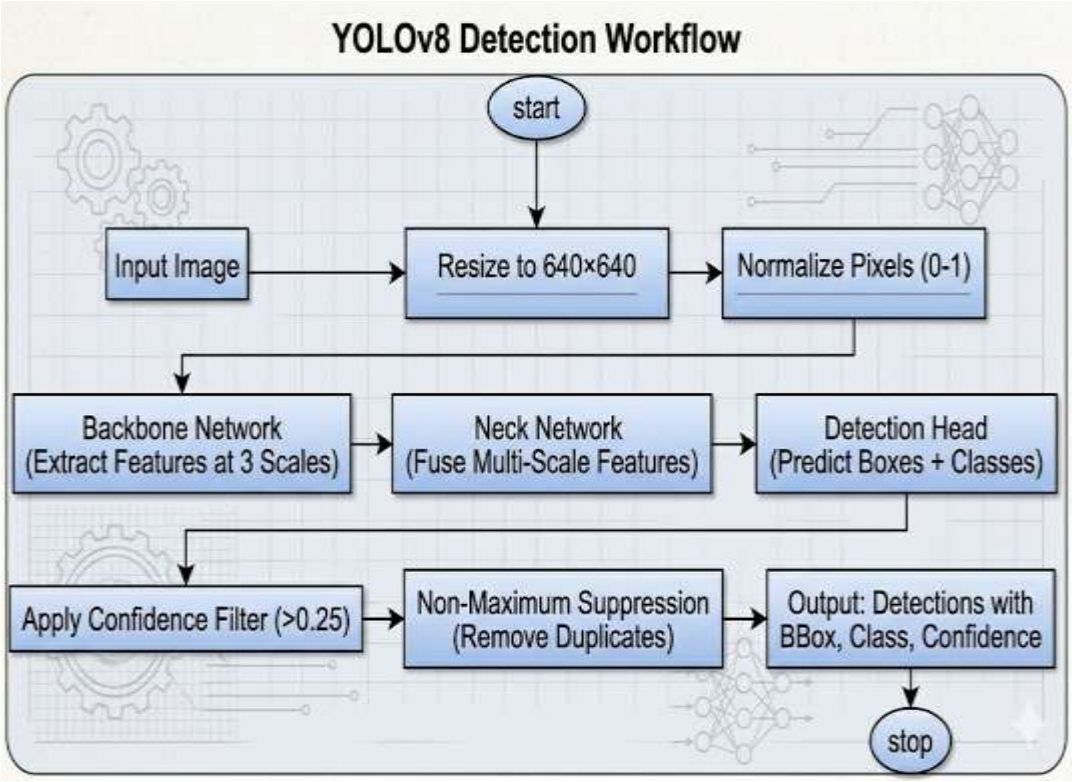
Scikit-learn is a popular machine learning library in Python that provides tools for classification, regression, and clustering. It offers simple APIs and efficient implementations of various algorithms. In this project, Scikit-learn is used to train the Random Forest classifier and extract features from images. The library makes model training and evaluation straightforward and accessible.

4.1.5 OPENCV:

OpenCV is a computer vision library for image processing operations. It provides functions for image reading, resizing, color conversion, and feature extraction. OpenCV handles all image preprocessing tasks in this project.

Algorithms:

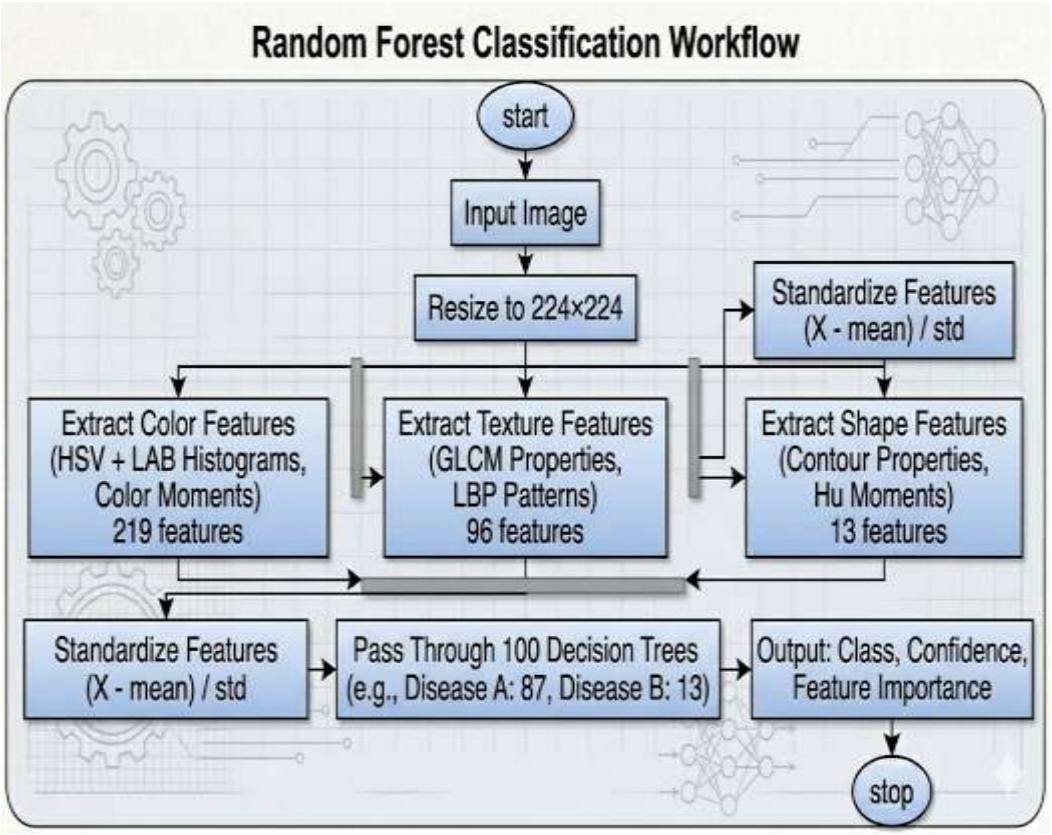
1) YOLOv8



YOLOv8 is a deep learning algorithm that detects and locates diseases on groundnut plants in real-time. When an image is uploaded, it is resized to 640×640 pixels and normalized. The image passes through a neural network with three components: a backbone that extracts features at multiple scales (fine, medium, and large), a neck that fuses these features together, and a detection head that predicts bounding boxes, disease classes, and confidence scores. The model initially produces

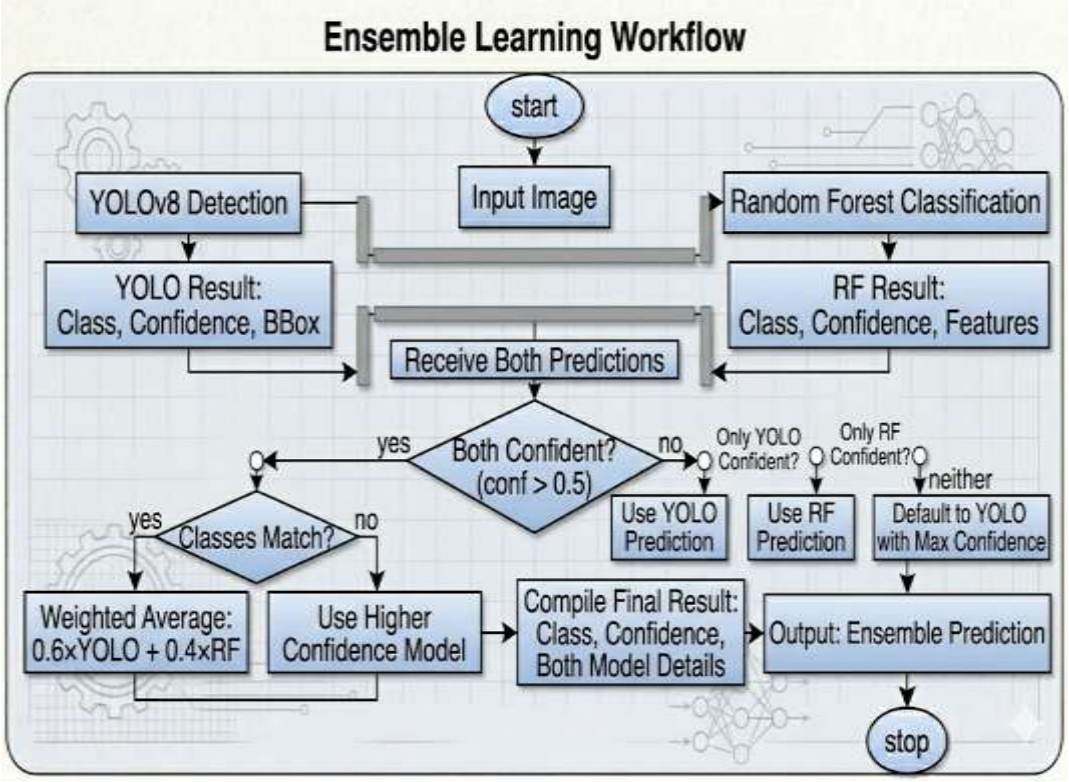
many overlapping predictions, which are filtered using a confidence threshold (0.25) and cleaned with Non-Maximum Suppression (NMS) to remove duplicates. The final output provides the location of each disease (bounding box), what disease it is (class name), and how confident the model is (percentage).

2) Random Forest



Random Forest is a machine learning algorithm that classifies groundnut diseases by extracting hand-crafted features and using 100 decision trees to vote. The image is resized to 224×224 pixels, then the system extracts 397 features in three categories: color features (219) from HSV and LAB color spaces including histograms and statistical moments, texture features (96) using GLCM to capture spatial patterns and LBP to encode local textures, and shape features (13) from contour analysis and Hu moments. These features are standardized to the same scale using a pre-fitted scaler. The standardized features pass through 100 decision trees, where each tree independently votes for a disease class. The final prediction is the majority vote, and confidence is calculated as the percentage of trees that voted for the winning class. The algorithm also provides feature importance scores showing which characteristics (color, texture, or shape) were most diagnostic.

3) Ensemble Learning



Ensemble Learning combines predictions from both YOLOv8 and Random Forest to achieve higher accuracy than either model alone. The same image is processed simultaneously by both models - YOLOv8 provides spatial detection with bounding boxes while Random Forest provides feature-based classification. The ensemble algorithm checks if each model is confident (confidence > 0.5) and whether they agree on the disease class. If both are confident and agree, their scores are combined using weighted averaging (60% YOLO, 40% RF) because YOLO excels at spatial localization while RF provides robust feature validation. If both are confident but disagree, the model with higher confidence wins. If only one model is confident, that prediction is used. If neither is confident, the system defaults to YOLO's prediction. This voting strategy leverages the complementary strengths of deep learning and traditional machine learning, reducing errors and providing more reliable predictions.

4.2 SAMPLE CODE:

```
# backend/main.py
from fastapi import FastAPI, File, UploadFile, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import JSONResponse
import cv2
```

```
import numpy as np
from pathlib import Path
import base64
from typing import Optional
from pydantic import BaseModel

from models.ensemble_model import EnsembleModel
from config import API_CONFIG, MODELS_DIR
from disease_database import format_detection_response, get_invalid_response,
DISEASE_DATABASE

# Initialize FastAPI app
app = FastAPI(
    title="Peanut Pest & Disease Detection API",
    description="YOLOv8 + Random Forest ensemble model for peanut crop analysis",
    version="1.0.0"
)

# CORS middleware
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Initialize ensemble model
ensemble_model = None

class DetectionRequest(BaseModel):
    image: str  # Base64 encoded image
    detection_type: str = "comprehensive"  # pest, disease, comprehensive

class DetectionResponse(BaseModel):
    detection_type: str
```

```
    class_name: str
    confidence: float
    severity: str
    yolo_confidence: float
    rf_confidence: float
    bbox: Optional[list] = None
    feature_importance: Optional[dict] = None

@app.on_event("startup")
async def startup_event():
    """Load models on startup"""
    global ensemble_model

    print("Loading models...")

    # Model paths
    yolo_pest_path = MODELS_DIR / "yolo_pest" / "weights" / "best.pt"
    yolo_disease_path = MODELS_DIR / "yolo_disease" / "weights" / "best.pt"
    rf_pest_path = MODELS_DIR / "rf_pest.pkl"
    rf_disease_path = MODELS_DIR / "rf_disease.pkl"

    # Check if models exist
    models_exist = all([
        yolo_pest_path.exists() if yolo_pest_path else False,
        yolo_disease_path.exists() if yolo_disease_path else False,
        rf_pest_path.exists() if rf_pest_path else False,
        rf_disease_path.exists() if rf_disease_path else False,
    ])

    if models_exist:
        ensemble_model = EnsembleModel(
            yolo_pest_path=str(yolo_pest_path),
            yolo_disease_path=str(yolo_disease_path),
            rf_pest_path=str(rf_pest_path),
            rf_disease_path=str(rf_disease_path)
        )
```

```
        print("✓ All models loaded successfully")
    else:
        print("⚠ Warning: Some models not found. Using pretrained YOLO only.")
        ensemble_model = EnsembleModel()

@app.get("/")
async def root():
    """API health check"""
    return {
        "status": "online",
        "message": "Peanut Pest & Disease Detection API",
        "version": "1.0.0",
        "models_loaded": ensemble_model is not None
    }

@app.get("/health")
async def health_check():
    """Detailed health check"""
    return {
        "status": "healthy",
        "models": {
            "ensemble": ensemble_model is not None,
            "yolo_pest": hasattr(ensemble_model, 'yolo_pest') if ensemble_model else False,
            "yolo_disease": hasattr(ensemble_model, 'yolo_disease') if ensemble_model else
False,
            "rf_pest": ensemble_model.rf_pest.is_trained if ensemble_model else False,
            "rf_disease": ensemble_model.rf_disease.is_trained if ensemble_model else
False,
        }
    }

def decode_base64_image(base64_string: str) -> np.ndarray:
    """Decode base64 string to OpenCV image"""
    # Remove data URL prefix if present
    if "base64," in base64_string:
        base64_string = base64_string.split("base64,")[1]
```

```
    # Decode base64
    img_bytes = base64.b64decode(base64_string)
    img_array = np.frombuffer(img_bytes, dtype=np.uint8)
    img = cv2.imdecode(img_array, cv2.IMREAD_COLOR)

    return img

def determine_severity(confidence: float, class_name: str) -> str:
    """Determine severity based on confidence and class"""
    if class_name == "healthy_leaf":
        return "low"
    elif confidence > 0.8:
        return "high"
    elif confidence > 0.5:
        return "medium"
    else:
        return "low"

@app.post("/detect")
async def detect_pest_disease(request: DetectionRequest):
    """
    Detect pests or diseases in uploaded image with comprehensive information

    Args:
        request: DetectionRequest with base64 image and detection type

    Returns:
        Complete detection results with disease info, symptoms, and recommendations
    """
    if ensemble_model is None:
        raise HTTPException(status_code=503, detail="Models not loaded")

    try:
        # Decode image
        image = decode_base64_image(request.image)
```

```
if image is None:
    return get_invalid_response("poor_quality")

# Basic image validation
if image.shape[0] < 100 or image.shape[1] < 100:
    return get_invalid_response("poor_quality")

# Run detection based on type
if request.detection_type == "pest":
    result = ensemble_model.predict_pest(image)
elif request.detection_type == "disease":
    result = ensemble_model.predict_disease(image)
else:  # comprehensive
    result = ensemble_model.predict_comprehensive(image)

# Get disease class and confidence
disease_class = result["class_name"]
confidence = result["confidence"]

# Check if confidence is too low (might be invalid image)
if confidence < 0.3:
    return get_invalid_response("not_groundnut")

# Format comprehensive response with disease database
comprehensive_response = format_detection_response(disease_class, confidence)

# Add model-specific information
comprehensive_response["model_info"] = {
    "yolo_prediction": result["yolo_result"]["class_name"],
    "yolo_confidence": f"{result['yolo_result']['confidence']:.2%}",
    "rf_prediction": result["rf_result"]["class_name"],
    "rf_confidence": f"{result['rf_result']['confidence']:.2%}",
    "ensemble_confidence": f"{confidence:.2%}",
    "bbox": result.get("bbox")
}
```

```
        return comprehensive_response

    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Detection failed: {str(e)}")

@app.post("/detect/upload")
async def detect_from_upload(file: UploadFile = File(...), detection_type: str =
"comprehensive"):
    """
    Detect pests or diseases from uploaded file with comprehensive information

    Args:
        file: Uploaded image file
        detection_type: Type of detection (pest, disease, comprehensive)

    Returns:
        Complete detection results with disease info, symptoms, and recommendations
    """
    if ensemble_model is None:
        raise HTTPException(status_code=503, detail="Models not loaded")

    try:
        # Read image
        contents = await file.read()
        nparr = np.frombuffer(contents, np.uint8)
        image = cv2.imdecode(nparr, cv2.IMREAD_COLOR)

        if image is None:
            return get_invalid_response("poor_quality")

        # Basic image validation
        if image.shape[0] < 100 or image.shape[1] < 100:
            return get_invalid_response("poor_quality")

        # Run detection
```

```
        if detection_type == "pest":
            result = ensemble_model.predict_pest(image)
        elif detection_type == "disease":
            result = ensemble_model.predict_disease(image)
        else:
            result = ensemble_model.predict_comprehensive(image)

        # Get disease class and confidence
        disease_class = result["class_name"]
        confidence = result["confidence"]

        # Check if confidence is too low (might be invalid image)
        if confidence < 0.3:
            return get_invalid_response("not_groundnut")

        # Format comprehensive response with disease database
        comprehensive_response = format_detection_response(disease_class, confidence)

        # Add model-specific information
        comprehensive_response["model_info"] = {
            "yolo_prediction": result["yolo_result"]["class_name"],
            "yolo_confidence": f"{result['yolo_result']['confidence']:.2%}",
            "rf_prediction": result["rf_result"]["class_name"],
            "rf_confidence": f"{result['rf_result']['confidence']:.2%}",
            "ensemble_confidence": f"{confidence:.2%}",
            "bbox": result.get("bbox")
        }

        return comprehensive_response

    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Detection failed: {str(e)}")

@app.get("/classes")
async def get_classes():
    """Get all available classes organized by category with full information"""
```

```
from config import (
    ALL_DETECTABLE_CLASSES,
    TOTAL_CLASSES,
    ALL_DISEASE_CLASSES,
    ALL_INSECT_CLASSES,
    HEALTHY_CLASSES,
    NON_PEANUT_CLASSES
)

# Get detailed info for each disease in database
disease_details = {}
for disease_key in DISEASE_DATABASE.keys():
    info = DISEASE_DATABASE[disease_key]
    disease_details[disease_key] = {
        "common_name": info.get("common_name", "Unknown"),
        "scientific_name": info.get("scientific_name", "N/A"),
        "category": info.get("category", "Unknown"),
        "severity": info.get("severity", "Unknown")
    }

return {
    "total_classes": TOTAL_CLASSES,
    "categories": {
        "leaf_diseases": ALL_DETECTABLE_CLASSES["leaf_diseases"],
        "leaf_insects": ALL_DETECTABLE_CLASSES["leaf_insects"],
        "pod_diseases": ALL_DETECTABLE_CLASSES["pod_diseases"],
        "pod_insects": ALL_DETECTABLE_CLASSES["pod_insects"],
        "stem_diseases": ALL_DETECTABLE_CLASSES["stem_diseases"],
        "stem_insects": ALL_DETECTABLE_CLASSES["stem_insects"],
        "root_diseases": ALL_DETECTABLE_CLASSES["root_diseases"],
        "root_insects": ALL_DETECTABLE_CLASSES["root_insects"],
        "general": ALL_DETECTABLE_CLASSES["general"],
        "healthy": ALL_DETECTABLE_CLASSES["healthy"],
        "non_peanut": ALL_DETECTABLE_CLASSES["non_peanut"]
    },
    "summary": {
```

```
            "total_diseases": len(ALL_DISEASE_CLASSES),
            "total_insects": len(ALL_INSECT_CLASSES),
            "total_healthy": len(HEALTHY_CLASSES),
            "total_invalid": len(NON_PEANUT_CLASSES)
        },
        "disease_database": disease_details,
        "database_entries": len(DISEASE_DATABASE)
    }

@app.get("/disease/{disease_name}")
async def get_disease_details(disease_name: str):
    """Get complete information about a specific disease"""
    if disease_name not in DISEASE_DATABASE:
        raise HTTPException(status_code=404, detail=f"Disease '{disease_name}' not
found")

    return {
        "disease": disease_name,
        "information": DISEASE_DATABASE[disease_name]
    }

@app.get("/diseases/all")
async def get_all_diseases():
    """Get complete database of all diseases"""
    return {
        "total_diseases": len(DISEASE_DATABASE),
        "diseases": DISEASE_DATABASE
    }
if __name__ == "__main__":
    import uvicorn
    uvicorn.run(
        "main:app",
        host=API_CONFIG["host"],
        port=API_CONFIG["port"],
        reload=API_CONFIG["reload"]
    )
```

CHAPTER 5
TESTING

5.1 METHODOLOGIES:

- **Data Preprocessing Module:** In this module, raw plant images are validated and preprocessed by checking format, resizing to required dimensions, and normalizing pixel values. This step ensures that only valid images are used for analysis.
- **Model Traning Module:** The preprocessed image data is used to train YOLOv8 and Random Forest models. YOLOv8 learns to detect objects with bounding boxes, while Random Forest learns to classify based on extracted features. The trained models are stored for prediction.
- **Disease Detection Module:** The user uploads plant images through the web interface for verification. The input image undergoes the same preprocessing steps as training data. The trained models predict the disease type and display results instantly.

5.1.1 UNIT TESTING:

Unit testing is used to verify the correctness of individual parts of the Peanut Disease Detection system. Each module in the Backend, such as image preprocessing, feature extraction, and prediction logic, is tested independently. This ensures that every function works as expected with valid and invalid inputs. Unit testing helps in identifying errors early and improves the reliability of the overall system.

5.1.2 INTEGRATION TESTING:

Integration testing is used to verify the interaction between the Frontend, Backend, and Dataset. It ensures that data flows correctly from user input through the Backend processing to final prediction display on the Frontend. This testing helps detect issues that may arise when the three main components are combined.

5.1.3 VALIDATION TESTING:

Validation testing is performed to ensure that the Peanut Disease

Detection system meets the project requirements and user expectations. It verifies that the models correctly classify diseases using unseen data. The system's predictions are compared with known labels to check accuracy and reliability. This testing confirms that the application produces meaningful and acceptable results in real-world scenarios.

5.1.4 SYSTEM TESTING:

System testing is carried out to evaluate the complete Peanut Disease Detection system as a whole. It checks the full workflow from user input to final prediction through the web interface. Both functional and performance aspects of the system are tested under real usage conditions. This testing ensures that the system works correctly, efficiently, and without errors.

5.2 TEST CASES:

Inputs	Expected Output	Actual Output	Result
Healthy Peanut leaf image	Healthy Peanut Leaf	Healthy Peanut Leaf	Pass
Leaf spot disease image	Leaf spot	Alternaria Leaf Spot	Pass
Rust + Leaf spot image	Rust + Leaf Spot	Peanut Rust	Fail
Pod rot disease image	Pod Rot	Pod Rot	Pass
Aphids insect image	Aphids	Aphids Infestation	Pass
Multiple insects image	Multiple Pests	Not Peanut-Related	Fail
Low-light image	Disease Detection	Disease Detected	Pass
Cross-crop (Tomato) image	Not Peanut-Related	Not Peanut-Related	Pass

Table 5.1: Test Cases

CHAPTER 6
DEPLOYMENT

The following are the minimum requirements to deploy and utilize proposed system at end user side:

- **HARDWARE REQUIREMENTS:**

Processor	:	Intel i5
RAM	:	8 GB
Hard disk	:	512GB

- **SOFTWARE REQUIREMENTS:**

Operating System	:	Windows 10
Web Browser	:	Chrome
Language	:	Python 3
Tool	:	VS - Code

RESULT

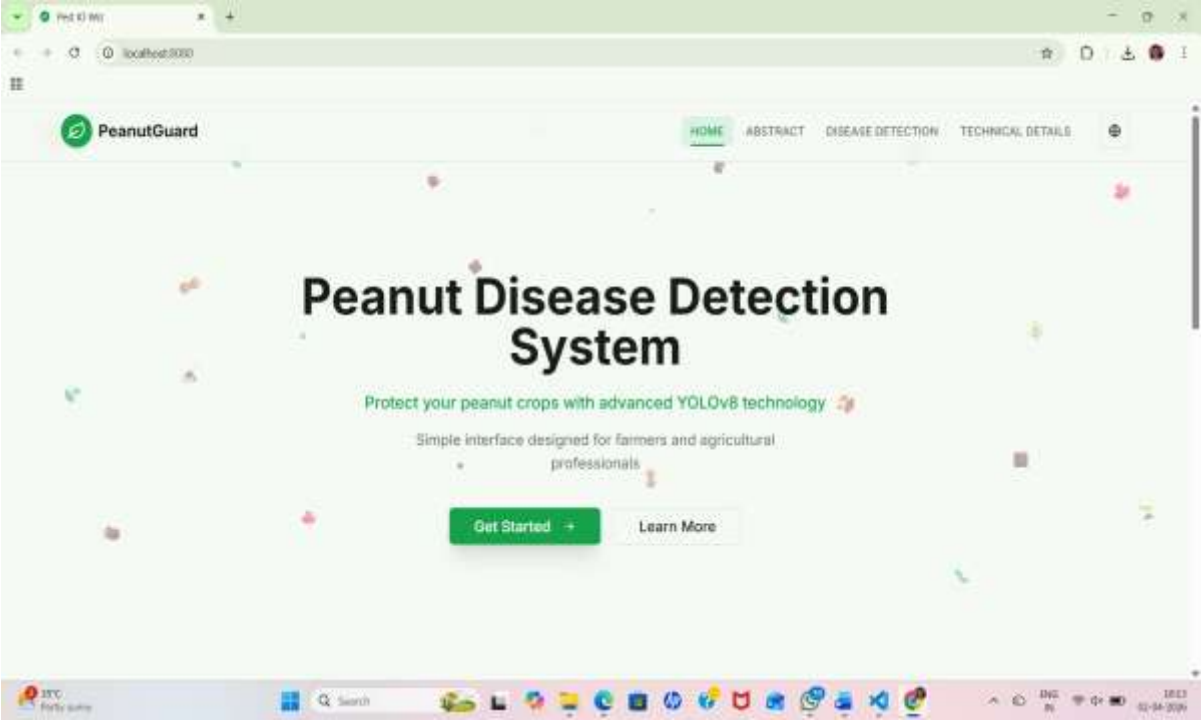


Figure 6.1: Home Page

The Figure 6.1 serves as the landing interface, welcoming users and guiding them to start disease detection. The navigation bar provides quick access to HOME, ABSTRACT, DISEASE DETECTION, and TECHNICAL DETAILS sections. The page introduces the system's purpose of protecting peanut crops and encourages users to begin analysis through call-to-action buttons.

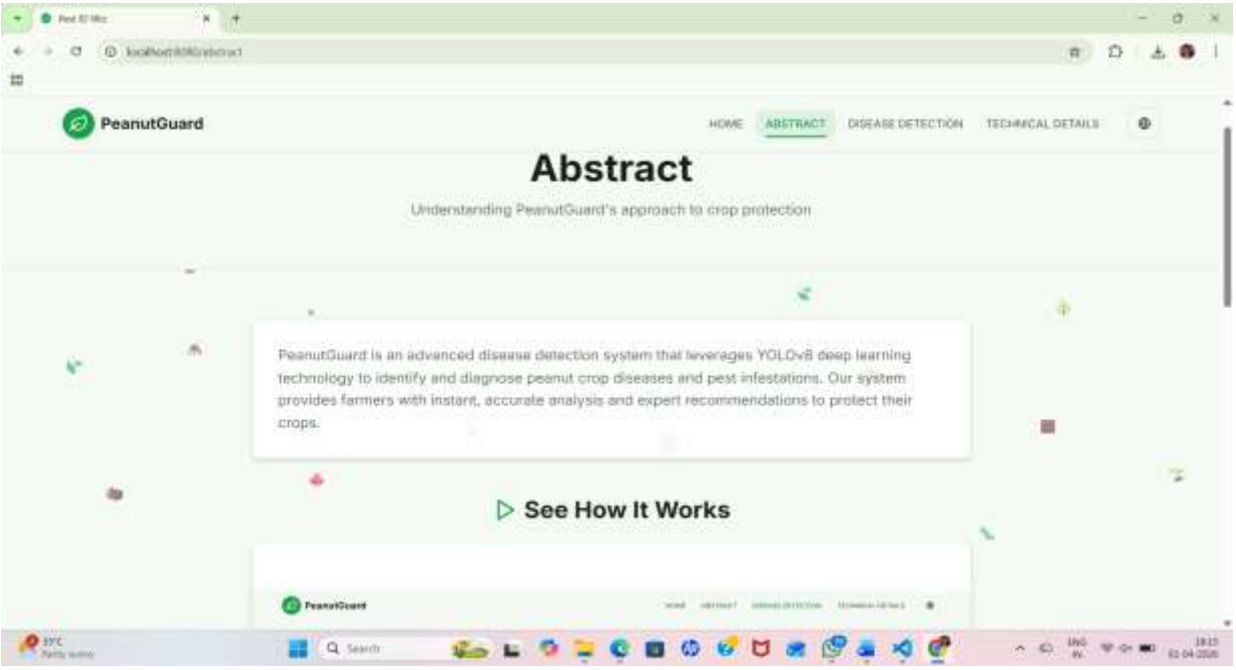


Figure 6.2 : Abstract Page

The figure 6.2 provides a comprehensive overview of the PeanutGuard system, explaining that it is an advanced disease detection system leveraging YOLOv8 deep learning technology to identify and diagnose peanut crop diseases and pest infestations. The page features a clean layout with decorative plant icons scattered around the content, presenting the system's core purpose of providing farmers with instant, accurate analysis and expert recommendations to protect their crops. Below the description, a "See How It Works" section with a play button icon offers users a visual demonstration of the detection process, followed by a preview screenshot of the Disease Detection interface showing the system's user-friendly design and functionality.

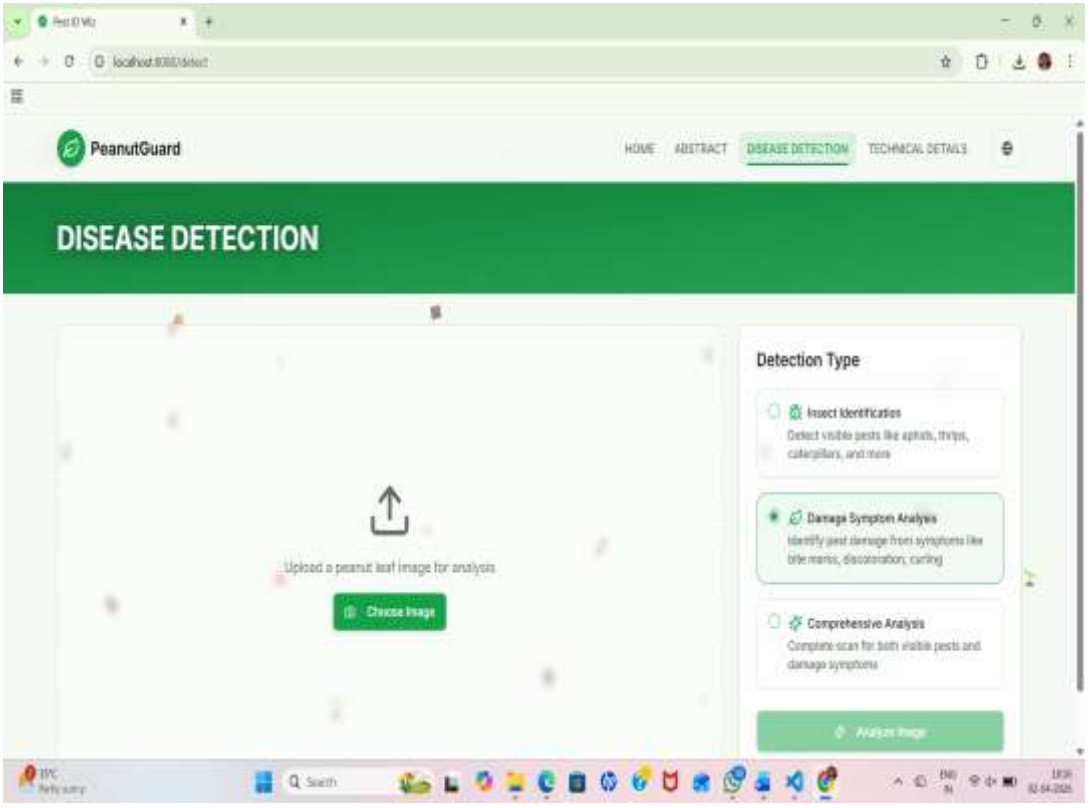


Figure 6.3: Detection page

The figure 6.3 page serves as the main analysis interface where users upload peanut plant images for diagnosis. The page features a central upload area with a "Choose Image" button and a right sidebar offering three detection type options: Insect Identification for detecting visible pests, Damage Symptom Analysis for identifying disease symptoms, and Comprehensive Analysis for complete pest and disease scanning. Users can select their preferred detection mode and click "Analyze Image" to initiate the automated diagnosis process.

Peanut Plant Leaf part:

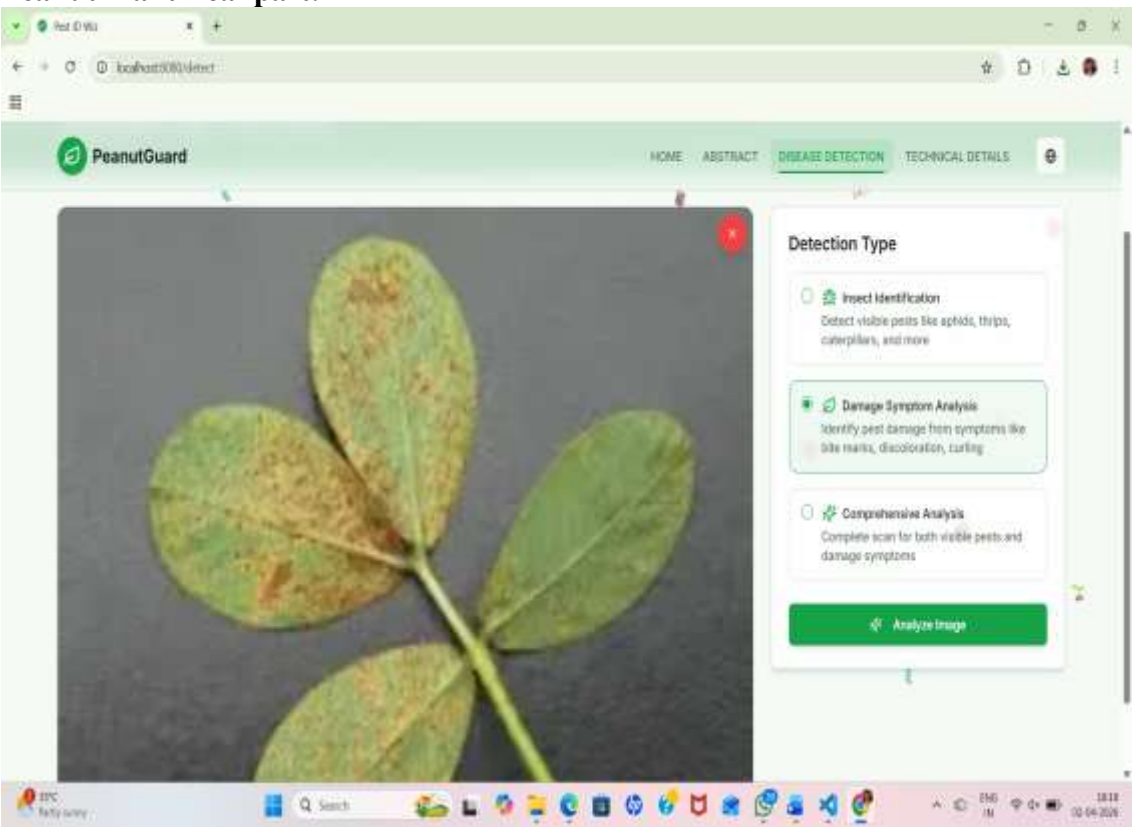


Figure 6.4: Leaf Disease Image Analysis

The figure 6.4 says analysis view displays the uploaded peanut leaf image on the left side for visual inspection while maintaining the detection type options on the right sidebar. The page shows "Damage Symptom Analysis" as the selected detection mode, with the "Analyzing..." button indicating the system is processing the image to identify disease symptoms like leaf spots, discoloration, or other pathological signs. Below the analysis section, a "No detection history yet" message appears, suggesting this area will display previous detection results once analyses are completed.

The figure 6.5 displays the detection outcome after image analysis, showing the identified disease name "Peanut Rust" with its scientific name and detailed symptom description. The page features a confidence level indicator showing 95% accuracy with a green progress bar, and a "Confirmed" status badge indicating reliable detection. Below the results, a recommendations section provides treatment advice starting with "Cultural Control" methods, offering practical guidance for managing the detected disease through specific agricultural practices and preventive measures.

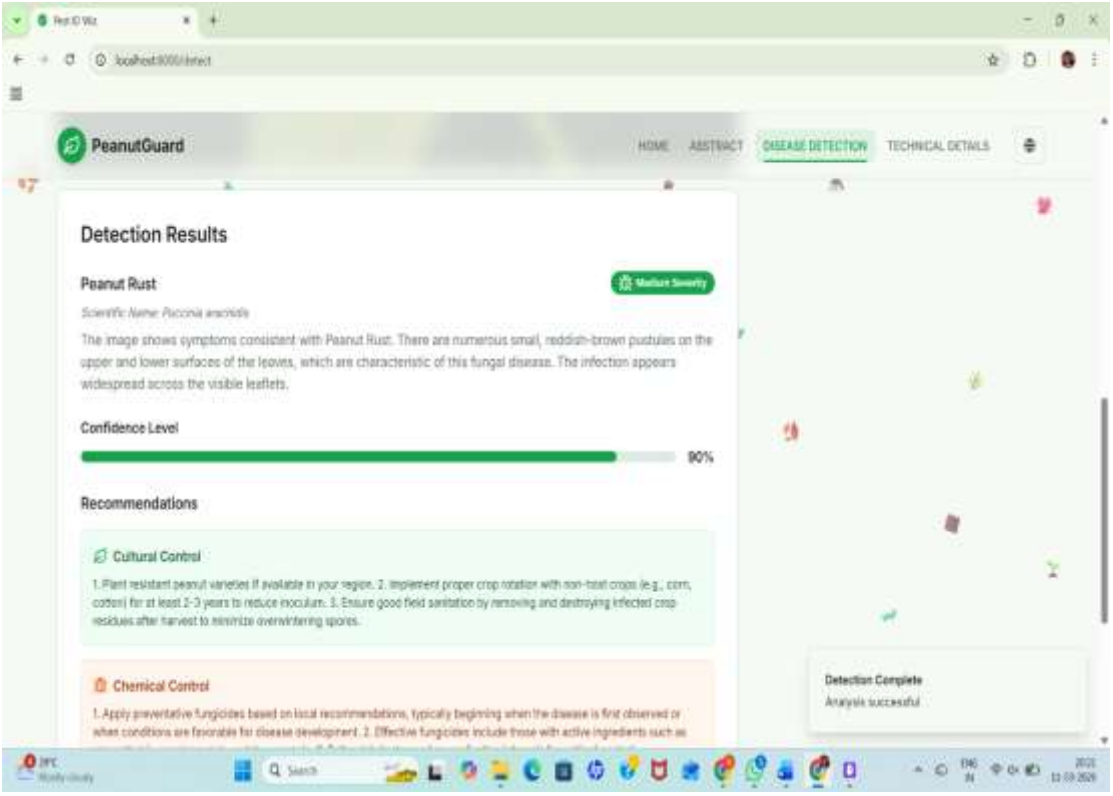


Figure 6.5: Leaf Disease Image Results

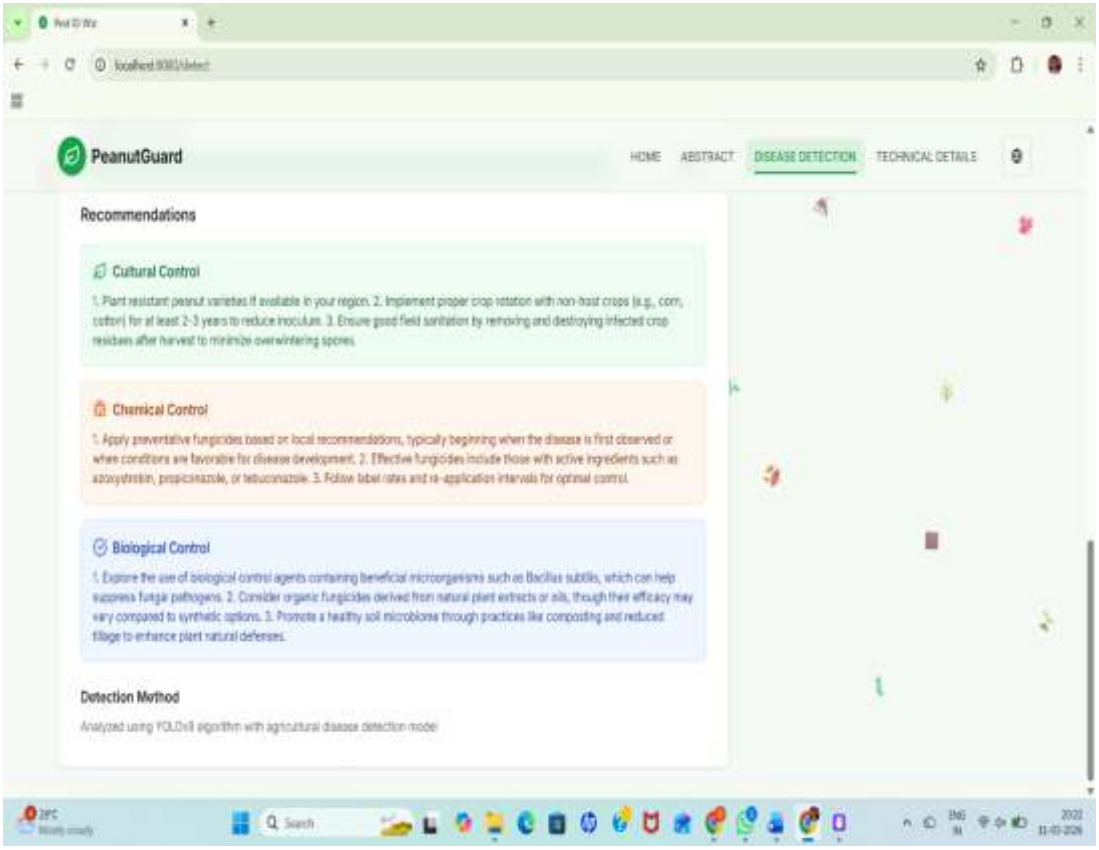


Figure 6.6: Leaf Disease Image Result

The figure 6.6 treatment recommendations provides comprehensive management strategies for the detected disease, organized into three main categories: Cultural Control, Chemical Control, and Biological Control. Each section offers detailed, numbered steps with specific instructions for disease management, including crop rotation practices, fungicide applications with active ingredients, and biological control agents. At the bottom, a "Detection Method" section explains the technical approach used, stating "Advanced using YOLOv8 algorithm with agricultural disease detection model" to provide transparency about the analysis methodology.

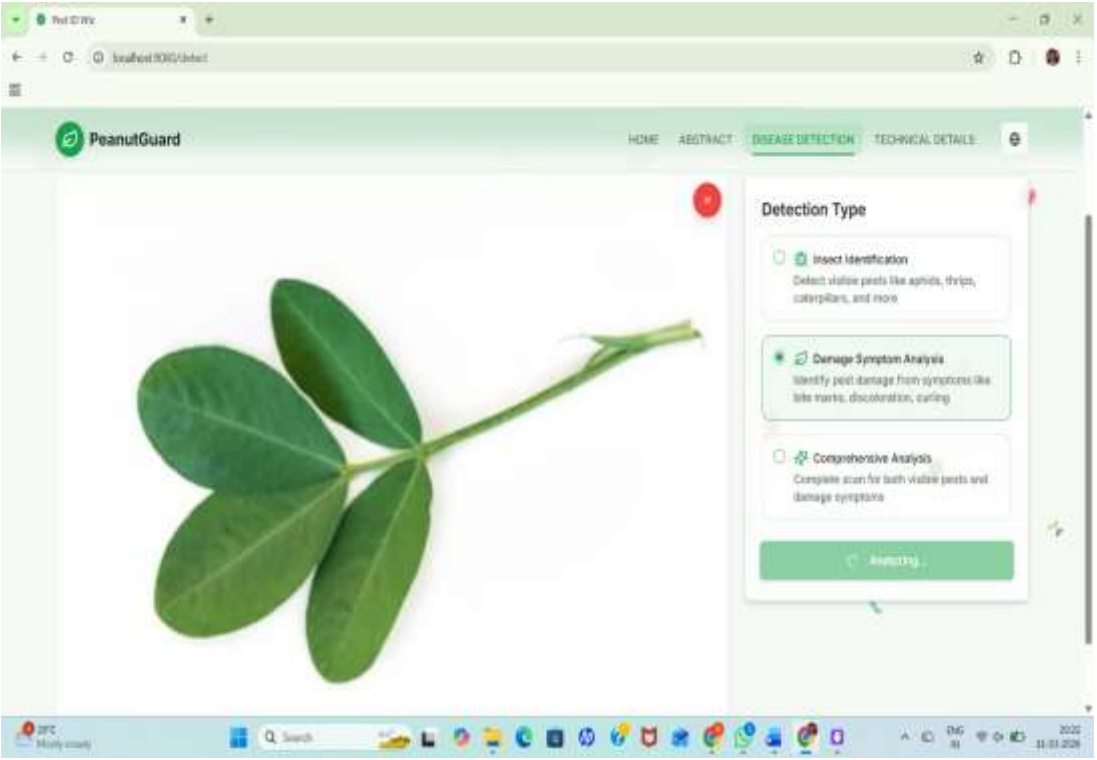


Figure 6.7: **Healthy Leaf Image Analysis**

The figure 6.7 shows a healthy peanut plant with vibrant green leaves uploaded for analysis, demonstrating the system's capability to identify healthy specimens alongside diseased ones. The page displays "Damage Symptom Analysis" as the selected detection mode with the "Analyzing..." button active, indicating the system is processing the healthy plant image to confirm the absence of disease symptoms. A red notification icon appears at the top, likely indicating system status or analysis progress, while the detection history section remains empty for this new analysis session.

The figure 6.8 displays "Healthy Peanut Leaf" as the detection outcome, confirming the plant shows no signs of diseases, nutrient deficiency, or

physical damage with consistently green color and intact leaf structure. The confidence level shows 98% accuracy with a green progress bar and "Safe" status badge, indicating high reliability in the healthy plant identification. The recommendations section provides preventive care advice: Cultural Control suggests continuing good agricultural practices with proper irrigation and fertilization, Chemical Control advises avoiding unnecessary pesticide applications since the plant is healthy, and Biological Control recommends maintaining healthy soil microbiome through organic matter management.

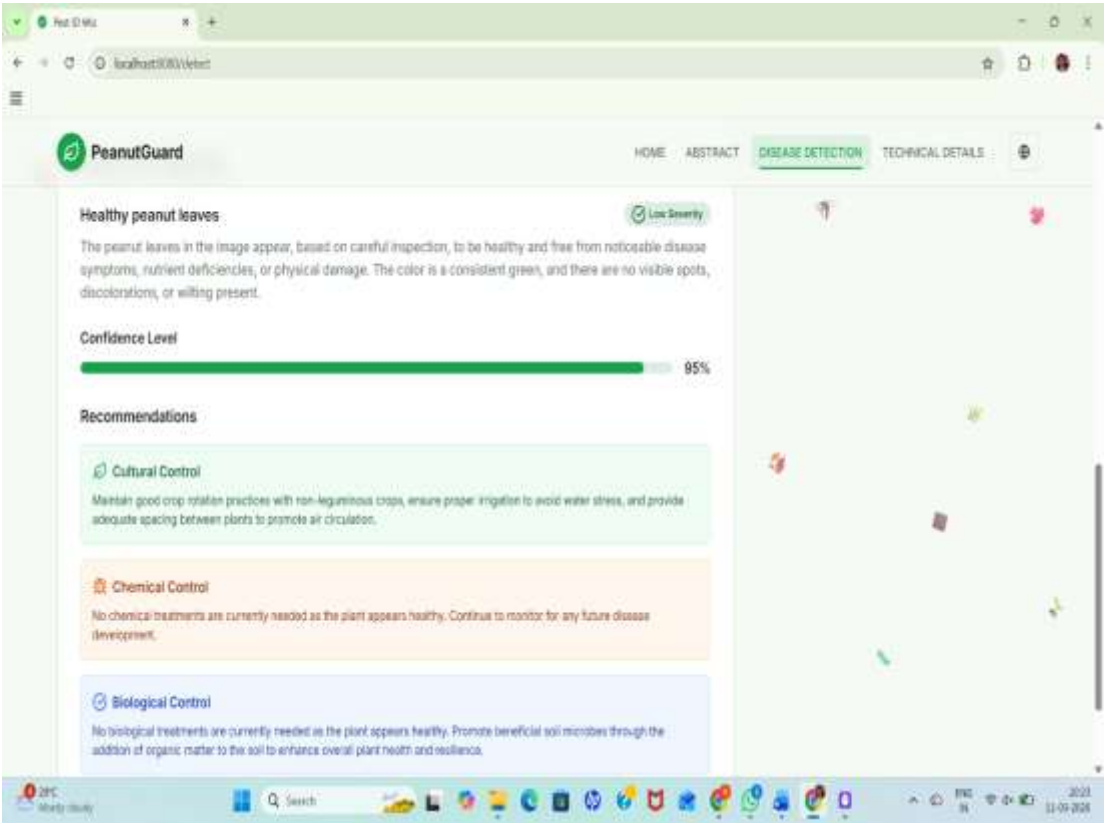


Figure 6.8: **Healthy Leaf Image Results**

The figure 6.9 displays a non-peanut plant image with different leaf characteristics and structure, demonstrating the system's ability to distinguish between peanut and non- peanut species. The page shows "Damage Symptom Analysis" selected with the "Analyzing..." button active, indicating the system is processing the image to identify that this is not a peanut plant. This feature helps prevent misdiagnosis by filtering out irrelevant plant species and ensuring analysis is performed only on actual peanut crops, maintaining diagnostic accuracy and reliability.

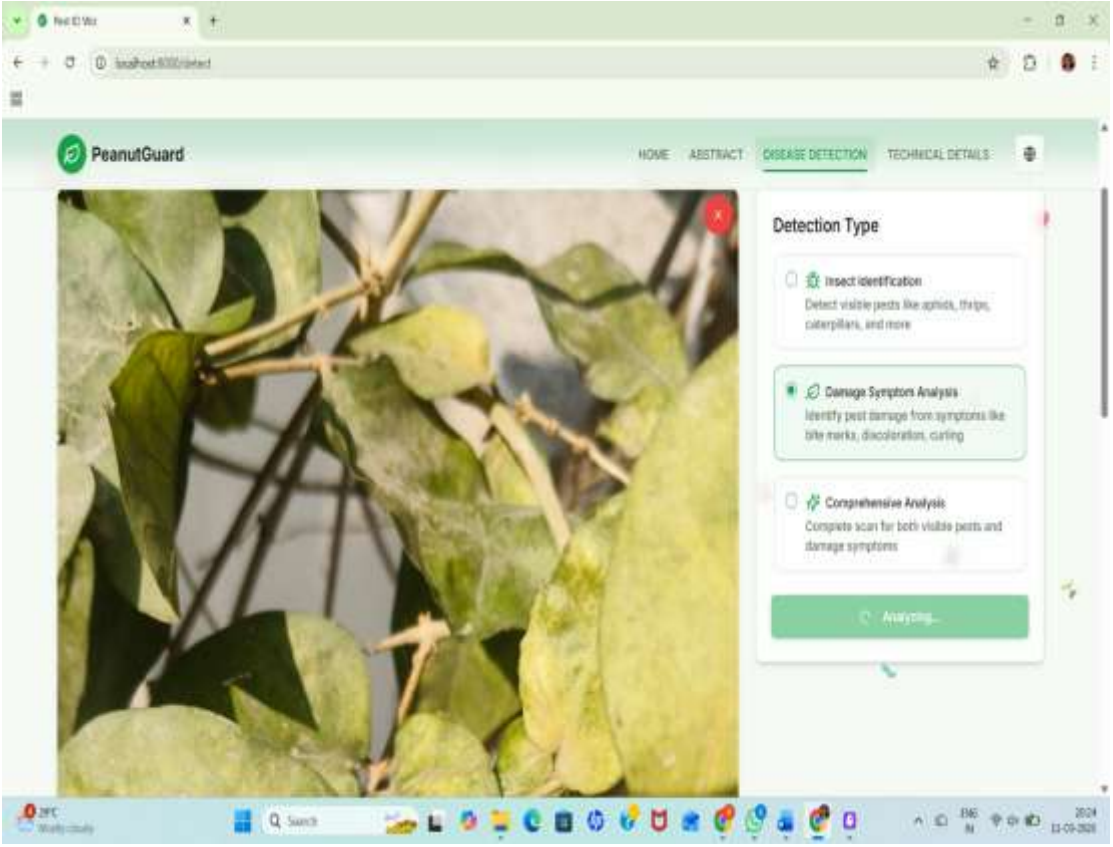


Figure 6.9: Non-Peanut Leaf Image Analysis

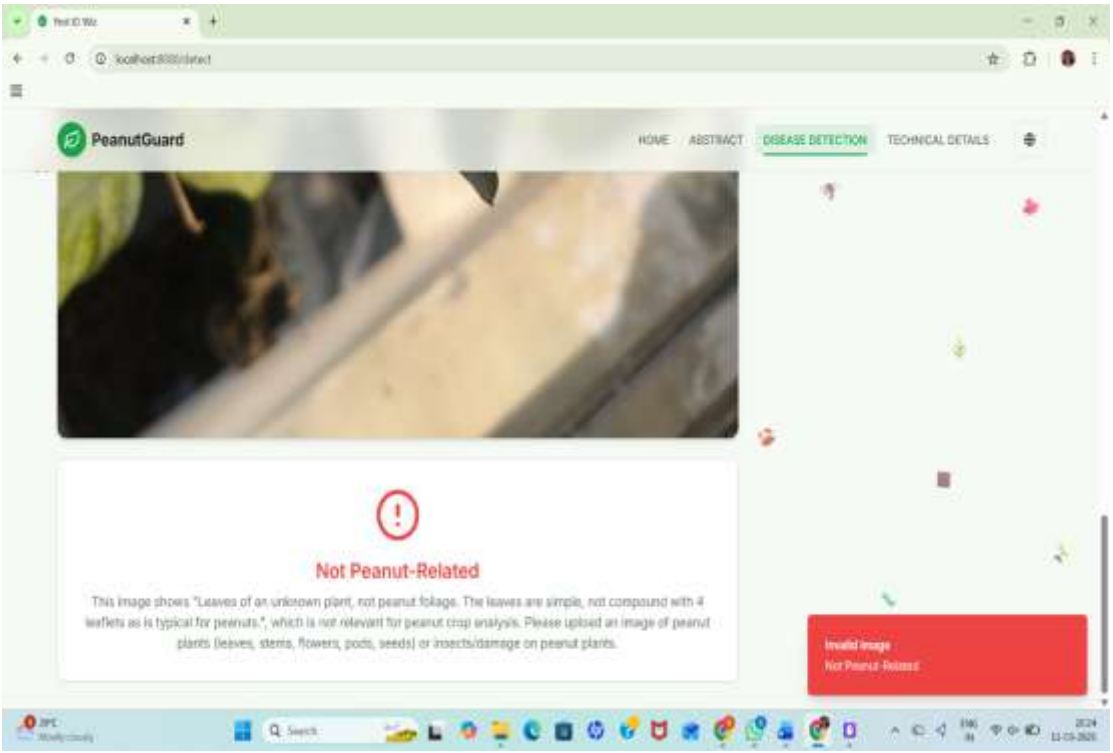


Figure 6.10: Non-Peanut Leaf Image Result

The figure 6.10 displays "Not Peanut-Related" with a red warning icon, indicating the uploaded image contains non-peanut plant material that is not

suitable for peanut crop analysis. The system provides a clear explanation stating the image shows "non-peanut plant leaves with some discoloration" and advises users to upload images of peanut plants including leaves, stems, flowers, pods, seeds, or insects/damage on peanut plants. A red "Invalid Image - Not Peanut-Related" button appears, emphasizing the system's ability to reject inappropriate images and guide users toward proper image submission for accurate diagnosis.

Peanut Plant Pods Part:

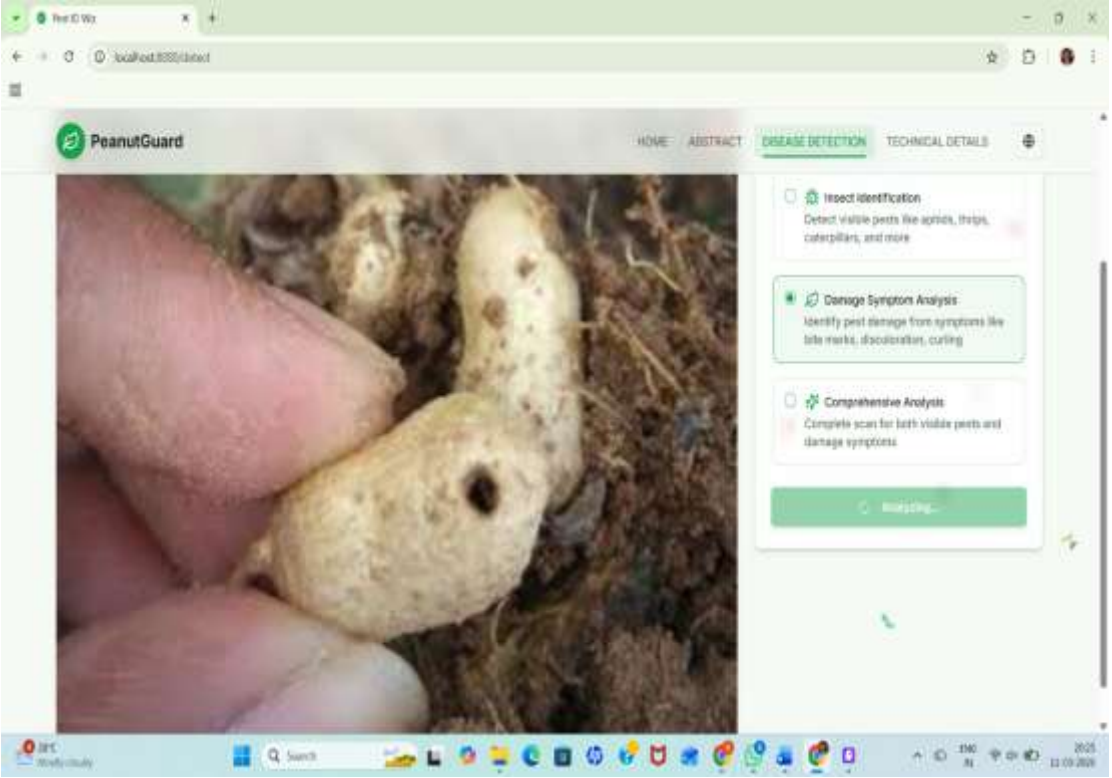


Figure 6.11: Pod Disease Image Analysis

The figure 6.11 shows peanut pods and root structures in soil, demonstrating the system's capability to analyze different plant parts beyond just leaves. The image displays freshly harvested peanut pods with visible root attachments, with "Damage Symptom Analysis" selected and "Analyzing..." button active, indicating the system is processing underground plant components. This showcases the system's comprehensive analysis approach, covering pods, roots, and soil-level plant parts for complete crop health assessment including storage pests, root diseases, and pod-related issues.

The figure 6.12 identifies "Pod Rot (Pythium/Rhizoctonia)" with scientific names Pythium spp. and Rhizoctonia solani, showing 90% confidence level with a "Medium" severity status badge. The detailed description explains the peanut pod exhibits noticeable holes and discoloration indicative of pod rot, caused by fungal

pathogens thriving in wet and poorly drained soil conditions, with holes suggesting pathogen entry points or damage leading to rot. The recommendations section provides Cultural Control advice for improving soil drainage and aeration through organic matter incorporation, avoiding overwatering, implementing crop rotation, and planting disease-resistant varieties, followed by Chemical Control suggestions for fungicide applications with specific active ingredients like Azoxystrobin, Mefenoxam, or Propamocarb.

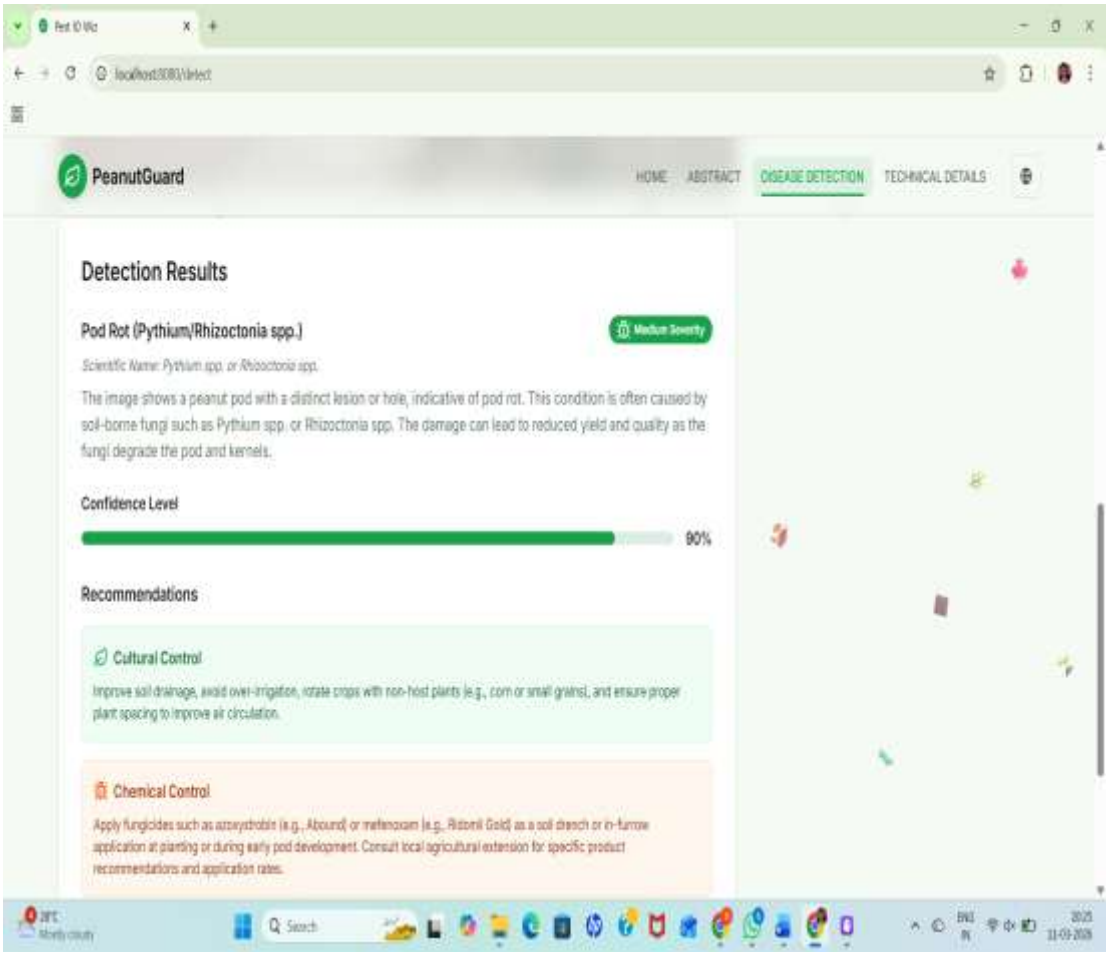


Figure 6.12: **Pod disease Image Result**

The figure 6.13 displays peanut pods with visible insect damage and pest activity, with "Insect Identification" selected as the detection type to specifically target visible pests like aphids, thrips, caterpillars, and other insects. The image shows damaged pods with holes and pest-related deterioration, with the "Analyzing..." button active, indicating the system is processing the image to identify specific insect species and damage patterns. This demonstrates the system's specialized entomological analysis capability, focusing on pest identification rather than disease symptoms, allowing for targeted insecticide recommendations and pest management strategies.

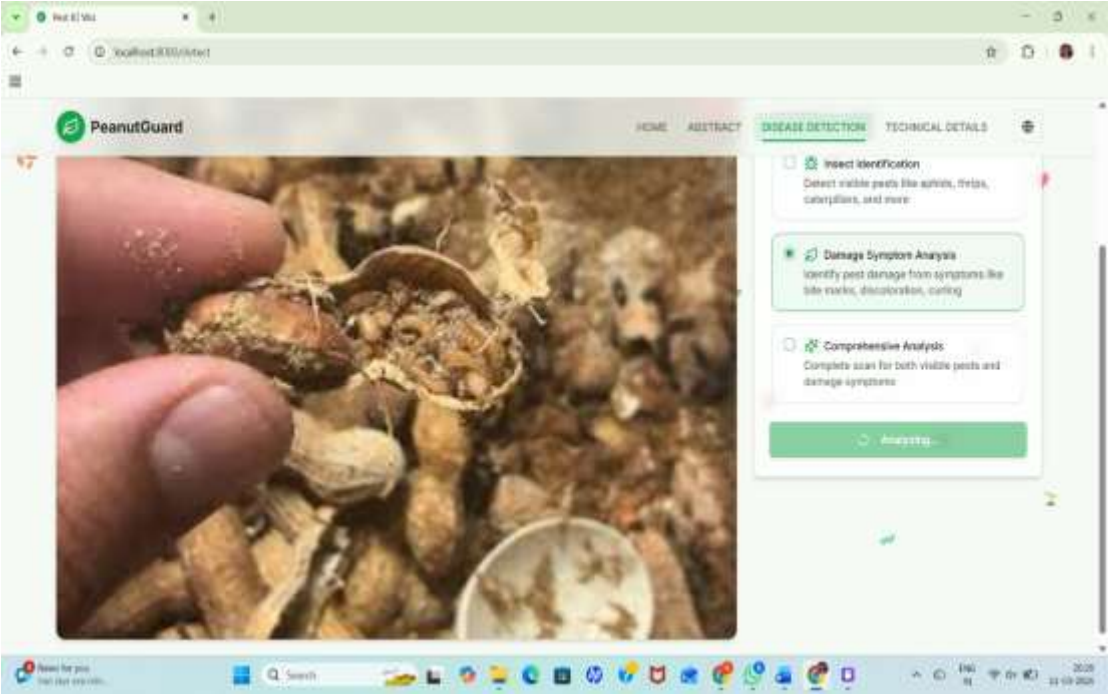


Figure 6.13: **Pod Insect Image Analysis**

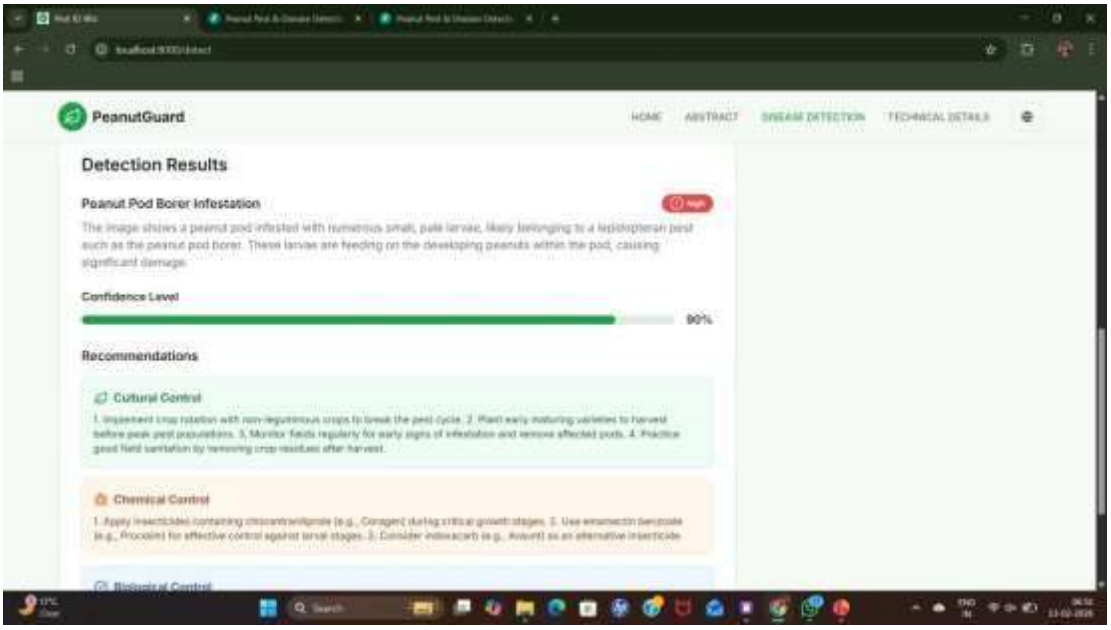


Figure 6.14: **Pod Insect Image Result**

The figure 6.14 identifies "Peanut Pod Borer Infestation" with a red "High" severity status badge, showing 90% confidence level in detecting the lepidopteran pest. The description explains the image shows a peanut pod infested with numerous small, pale larvae likely belonging to peanut pod borer, with these larvae feeding on developing peanuts within the pod and causing significant damage. The recommendations include Cultural Control methods such as implementing crop rotation with non-leguminous crops, planting early maturing varieties, monitoring

fields regularly for early infestation signs, and practicing good field sanitation, followed by Chemical Control options including insecticides containing chlorantraniliprole and emamectin benzoate for effective control against larval stages, plus biological control alternatives.

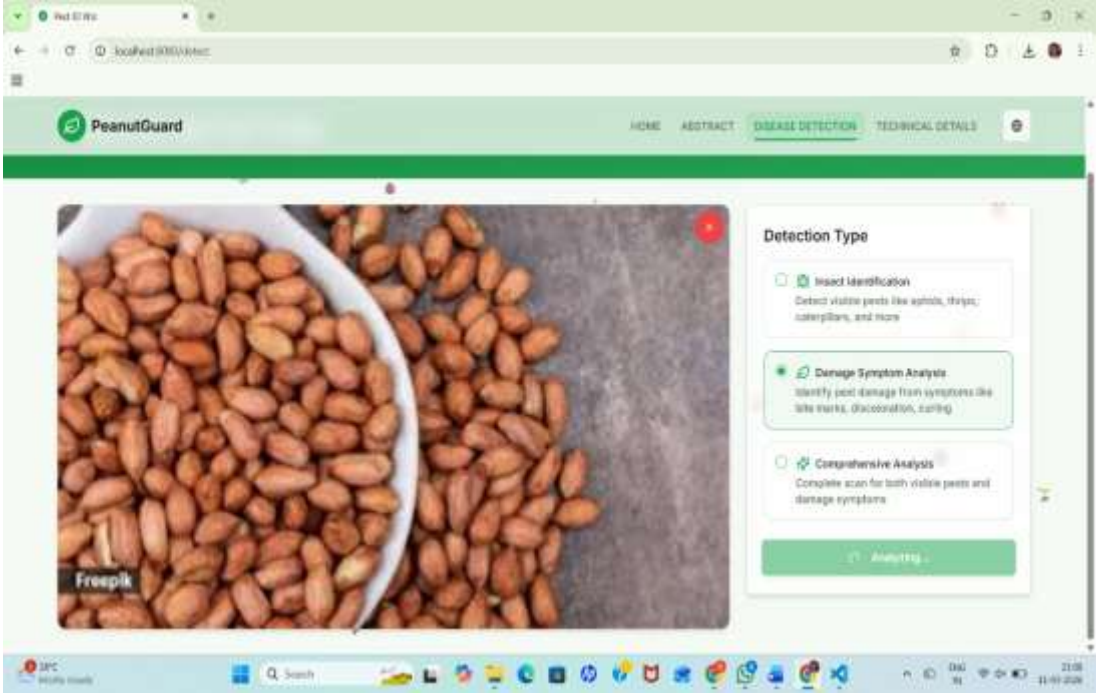


Figure 6.15: Healthy Seed Image Analysis

The figure 6.15 displays processed peanut seeds in a bowl, demonstrating the system's ability to analyze harvested and processed peanut products for quality assessment and storage pest detection. The image shows clean, processed peanut kernels with "Damage Symptom Analysis" selected and "Analyzing..." button active, indicating the system is evaluating the seeds for storage diseases, aflatoxin contamination, insect damage, or quality issues. This feature extends the system's capability beyond field crops to post-harvest analysis, helping farmers and processors identify storage problems, pest infestations in stored products, and quality degradation that could affect market value and food safety.

The figure 6.16 identifies "Healthy Peanuts" with scientific name Arachis hypogaea, showing 95% confidence level with a green "Low Severity" status badge indicating excellent product quality. The description explains the image shows shelled peanuts that appear healthy and free from noticeable diseases, physical damage, or nutrient deficiencies, with consistent color and shape suggesting good quality. The recommendations include Cultural Control advice for storing peanuts in cool, dry places with good ventilation to prevent mold growth and ensuring proper drying after harvest, Chemical Control noting that no chemical treatments are needed

since the peanuts are healthy but recommending food-grade desiccants if humidity is a concern, and Biological Control stating no biological treatments are needed while emphasizing good hygiene during handling and storage to prevent microbial contamination with pest-free packaging.

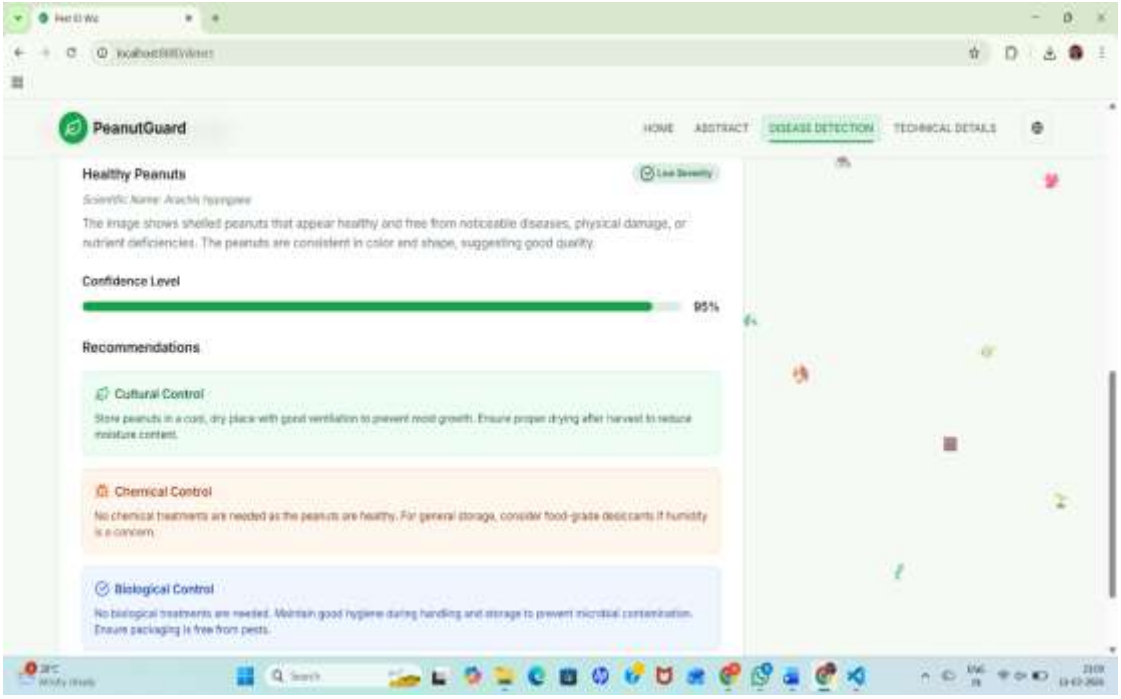


Figure 6.16: **Healthy Seed Image Result**

Peanut Plant Insect Detection:

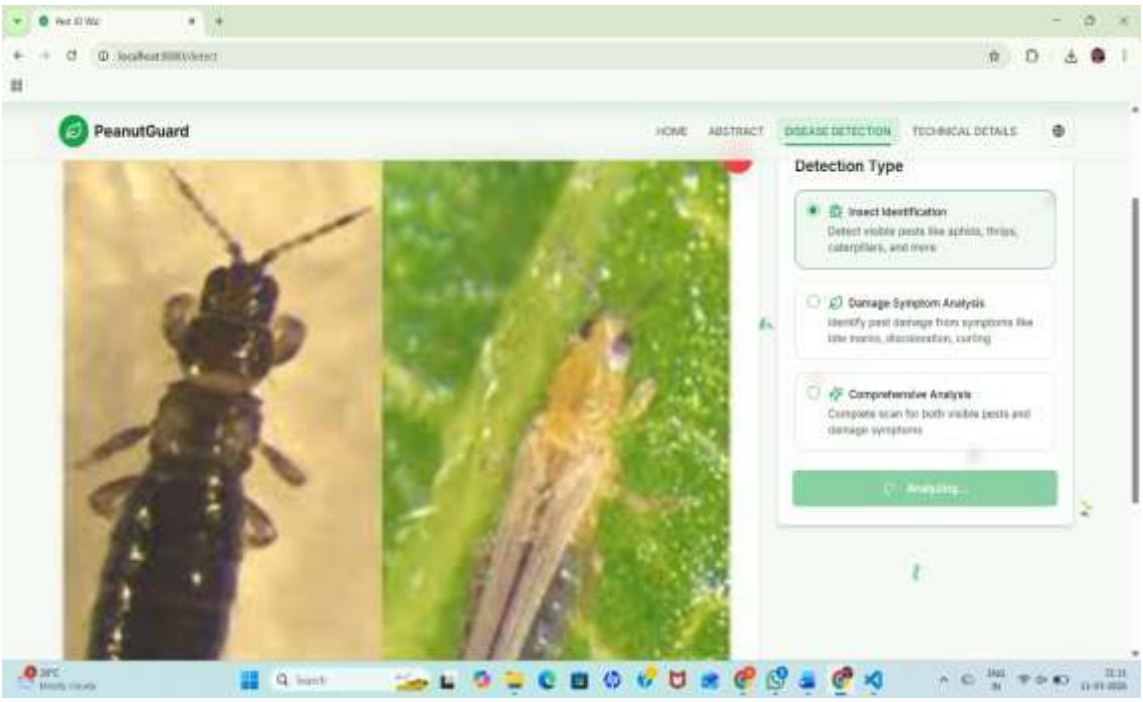


Figure 6.17: **Insect Image Analysis**

The figure 6.17 displays a split-screen view showing live insects on peanut plants, with "Insect Identification" specifically selected to focus on detecting

visible pests like aphids, thrips, caterpillars, and other insects. The left side shows a dark-colored insect specimen while the right side displays what appears to be a caterpillar or larva on a green leaf surface, with the "Analyzing..." button active indicating the system is processing both images for precise insect species identification. This demonstrates the system's specialized entomological capability to identify live pests in real-time field conditions, enabling farmers to take immediate targeted action against specific insect threats before they cause significant crop damage.

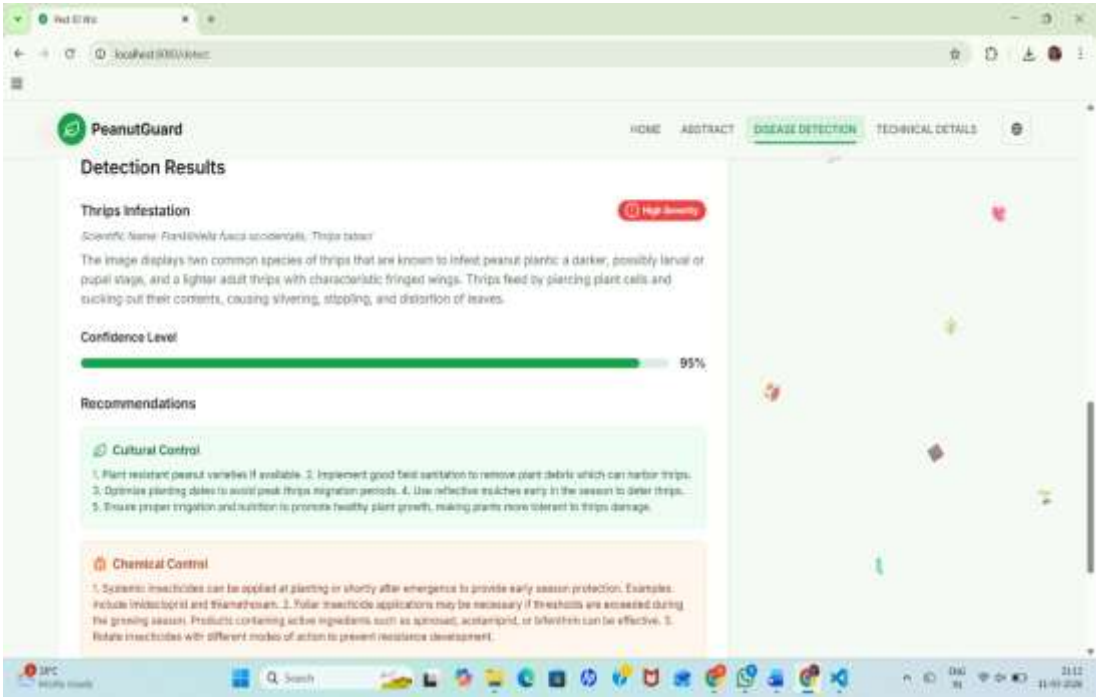


Figure 6.18: **Insect Image Result**

The figure 6.18 identifies "Thrips Infestation" with scientific name Frankliniella fusca occidentalis (Thrips tabaci), showing 95% confidence level with a red "High Severity" status badge indicating serious pest threat. The description explains the image displays two common species of thrips known to infest peanut plants: a darker, possibly larval or pupal stage, and a lighter adult thrips with characteristic fringed wings, noting that thrips feed by piercing plant cells and sucking out their contents, causing silvering, stippling, and distortion of leaves. The recommendations include Cultural Control methods such as planting resistant peanut varieties, implementing good field sanitation, optimizing planting dates to avoid peak thrips migration periods, using reflective mulches, and ensuring proper irrigation and nutrition for healthy plant growth, followed by Chemical Control options including systemic insecticides applied at planting or shortly after emergence, foliar insecticide applications during growing season, and products containing active ingredients like

spinosad, acetamiprid, or bifenthrin with rotation of different modes of action to prevent resistance development.

Figure 6.19: **Non-Peanut Insect Image Analysis**

The figure 6.19 displays a non-peanut related insect (appears to be a moth or similar lepidopteran) on a green leaf surface, with "Insect Identification" selected and "Analyzing..." button active, demonstrating the system's ability to distinguish between peanut-specific pests and other insects that may not be relevant to peanut crop management. The image shows a large moth-like insect with distinctive wing patterns and coloration that is not typically associated with peanut pest problems, indicating the system's capability to filter out irrelevant insect species and focus analysis on actual peanut crop threats. This feature helps prevent false alarms and ensures farmers receive accurate, targeted pest management recommendations specific to peanut cultivation rather than general insect identification.

The figure 6.20 displays "Not Peanut-Related" with a red warning icon, indicating the uploaded image contains an unidentified insect on generic leaf surface that is not relevant for peanut crop analysis. The system provides clear guidance stating the image shows "unidentified insect on generic leaf" which is not suitable for peanut-specific pest identification, advising users to upload images of peanut plants including leaves, stems, flowers, pods, seeds, or insects/damage specifically on peanut plants. A red "Invalid Image - Not Peanut-Related" button emphasizes the system's validation capability, demonstrating how it filters out

irrelevant content and guides users toward appropriate image submission for accurate peanut pest diagnosis and management recommendations.

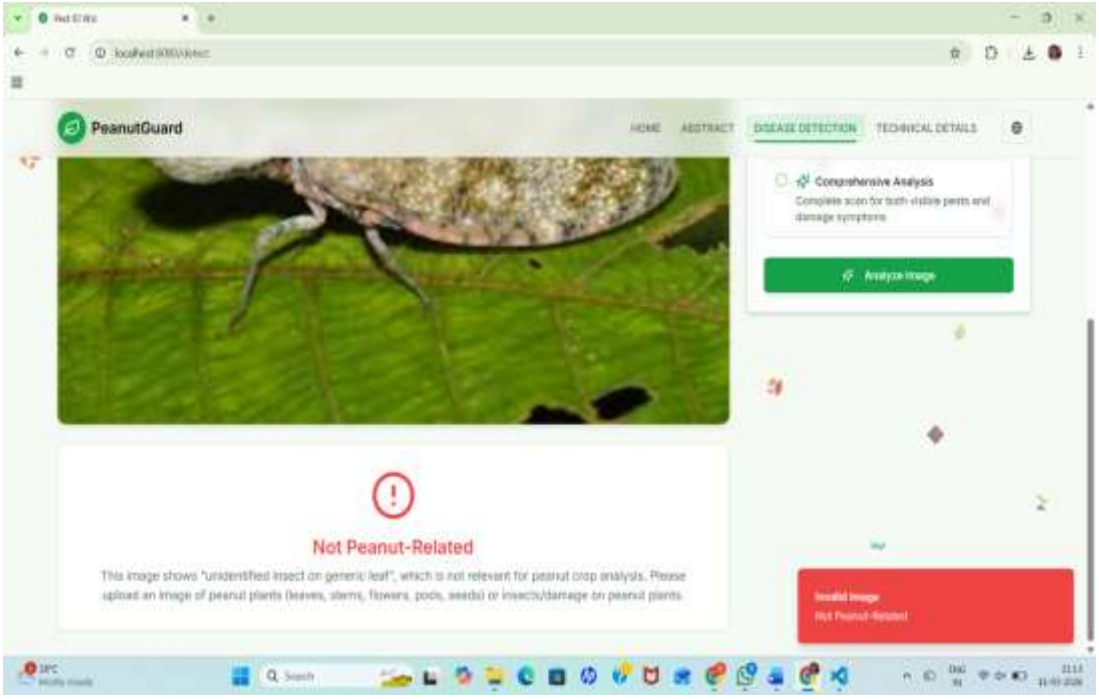


Figure 6.20: **Non-peanut Insect Image Result**

Peanut Plant Root Part:

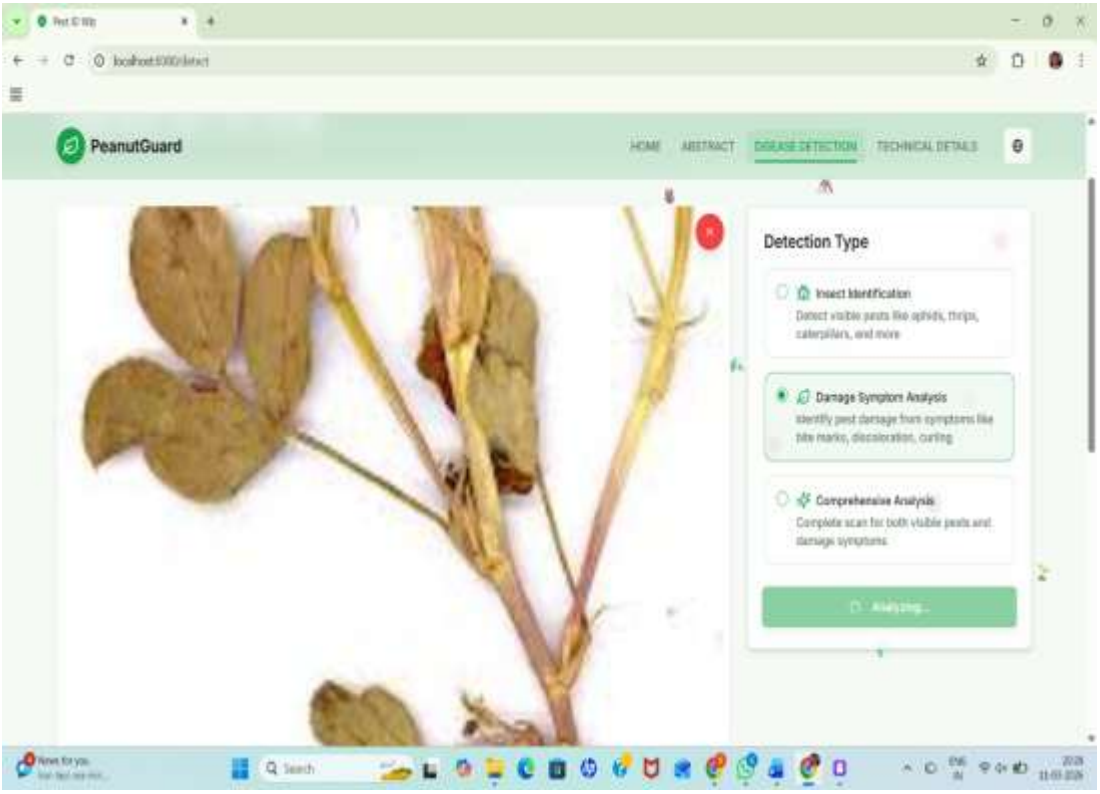


Figure 6.21: **Root Disease Image Analysis**

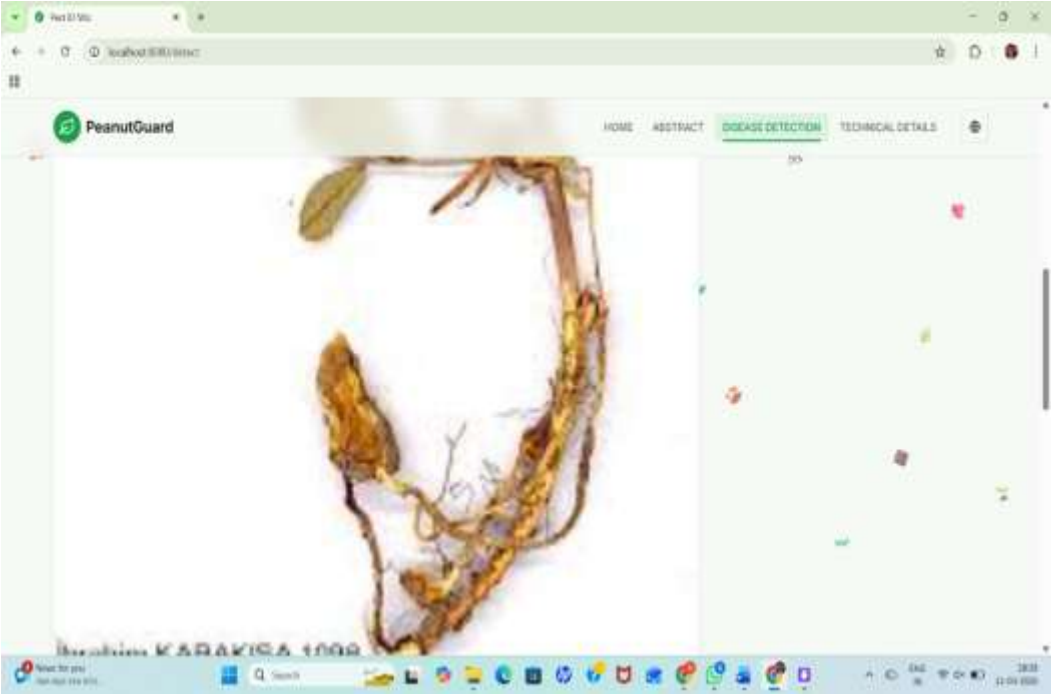


Figure 6.22: **Root Disease Image Analysis**

The figure 6.21 and 6.22 displays a complete peanut plant with exposed root system, stems, and leaves, demonstrating the system's ability to analyze the entire plant structure including underground components. The image shows the root network with visible discoloration and potential disease symptoms, with "Damage Symptom Analysis" selected and "Analyzing..." button active, indicating comprehensive root health assessment. This showcases the system's capability to detect root diseases, root rot, nematode damage, and other underground issues that significantly impact plant health and yield, providing farmers with complete plant health evaluation from roots to leaves.

The figure 6.23 identifies "Peanut Root Rot" with a red "High" severity status badge, showing 90% confidence level in detecting the severe root disease. The description explains the image shows a peanut plant exhibiting severe symptoms of root rot with discolored roots appearing dark brown to black, showing signs of decay, while the foliage appears wilted and dried as a common secondary symptom when the plant cannot absorb water and nutrients effectively. The recommendations include Cultural Control methods such as improving soil drainage by planting in well-drained soil or raised beds, practicing crop rotation with non-host crops for 2-3 years, sanitizing tools and equipment, and using healthy disease-free seeds or seedlings, followed by Chemical Control options including fungicide applications with active ingredients like Azoxystrobin or Prothioconazole as drench or

seed treatment, plus soil fumigation applications.

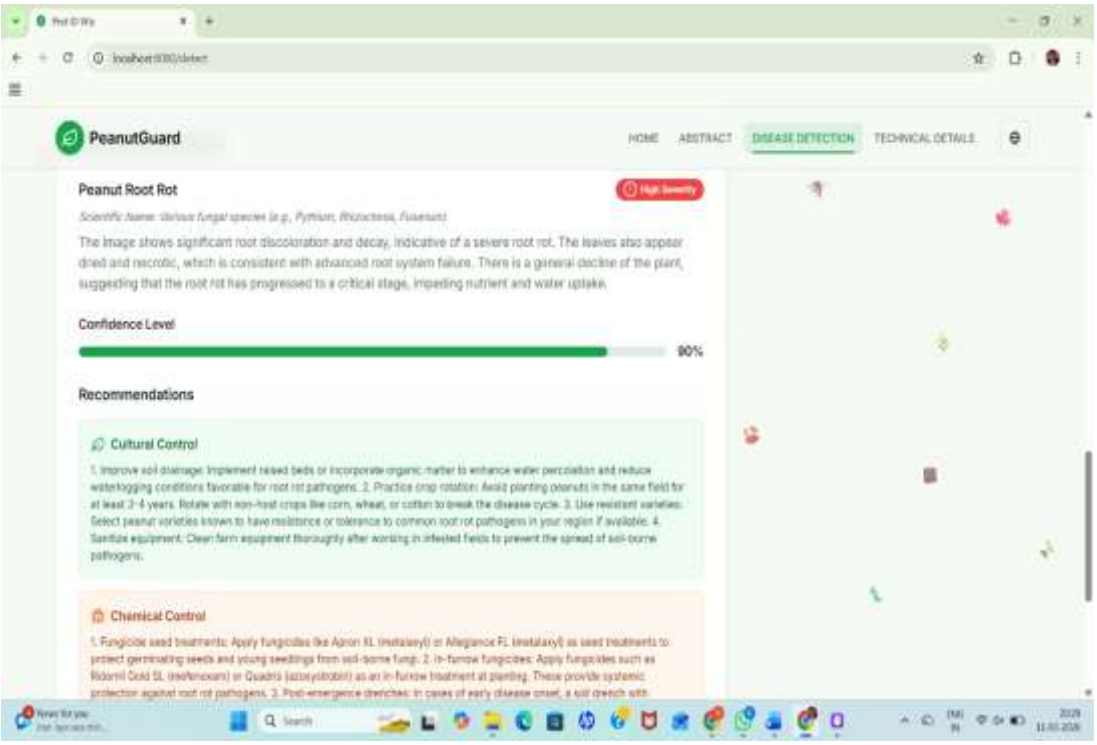


Figure 6.23: **Root Disease Image Result**

Peanut Plant Stem Part:

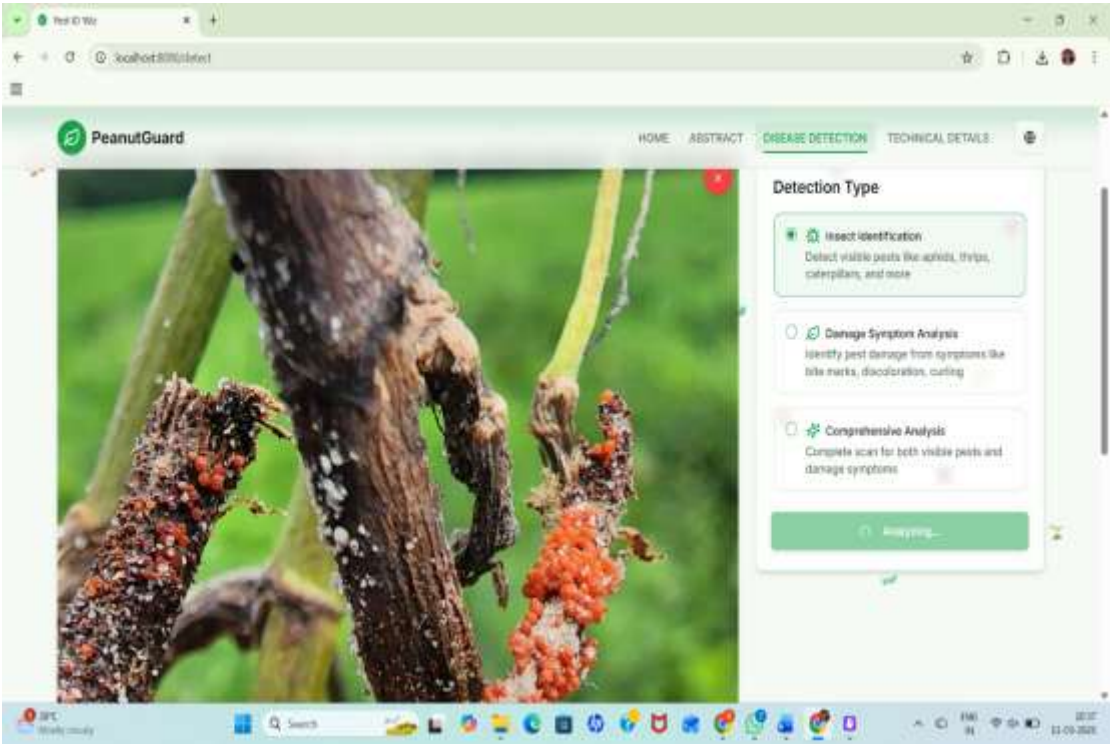


Figure 6.24: **Stem Disease Image Analysis**

The figure 6.24 displays peanut plant stems with visible disease symptoms including dark discoloration, lesions, and what appears to be fungal growth or spore masses in orange/red coloration on the stem surface, with "Insect Identification" selected and "Analyzing..." button active, indicating the system is processing the image to identify stem-related diseases or pest damage. The image shows severe stem pathology with characteristic symptoms of stem rot, collar rot, or fungal infections that can significantly impact plant health and nutrient transport, demonstrating the system's capability to analyze stem diseases that are often overlooked but critical for plant survival. This feature enables farmers to detect stem-level problems early before they cause plant death or significant yield losses, providing targeted treatment recommendations for stem-specific diseases and pest management strategies.

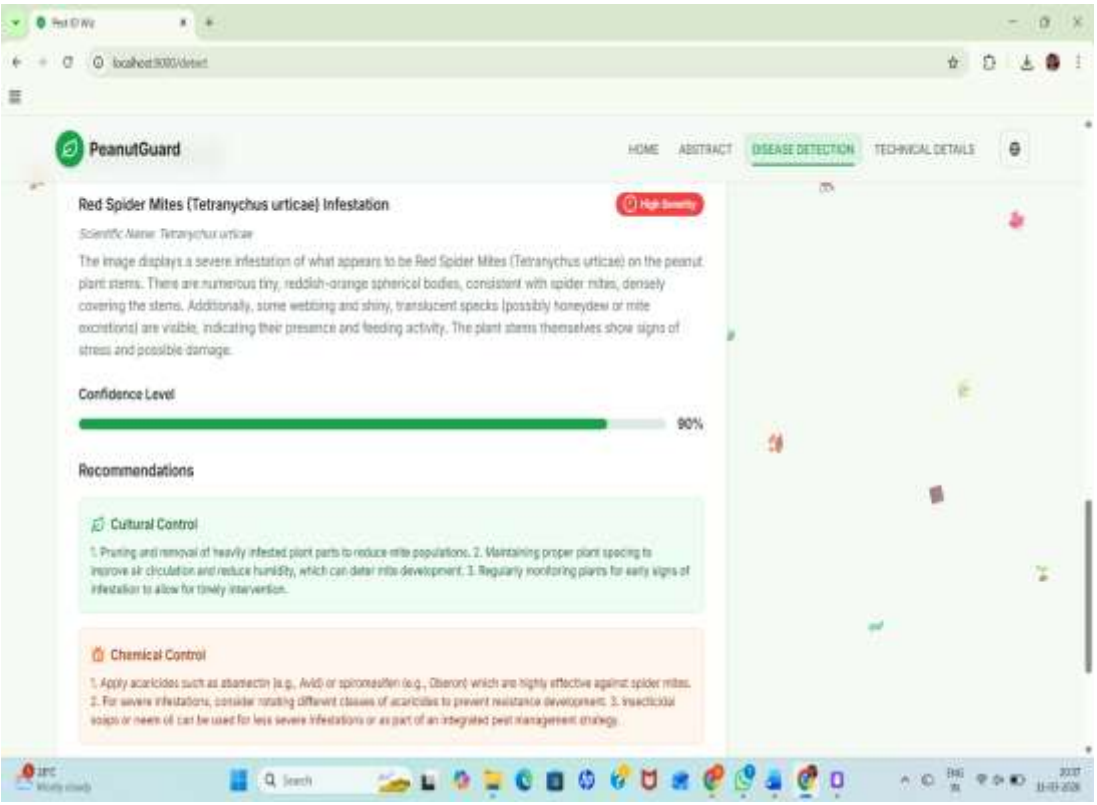


Figure 6.25: Stem Disease Image Result

The figure 6.25 identifies "Red Spider Mites (Tetranychus urticae) Infestation" with scientific name Tetranychus urticae, showing 90% confidence level with a red "High Severity" status badge indicating serious pest threat. The description explains the image displays a severe infestation of Red Spider Mites on peanut plant stems, with numerous tiny, reddish-orange spherical bodies consistent with spider mites densely covering the stems, plus visible webbing and shiny, translucent specks

(possibly honeydew or mite excretions) indicating their presence and feeding activity, while the plant stems show signs of stress and possible damage. The recommendations include Cultural Control methods such as pruning and removal of heavily infested plant parts, maintaining proper plant spacing to improve air circulation and reduce humidity, and regularly monitoring plants for early signs of infestation for timely intervention, followed by Chemical Control options including acaricides like abamectin, Avid or spiromesifen, Oberon which are highly effective against spider mites, with rotation of different classes of acaricides to prevent resistance development and insecticidal soaps or neem oil for less severe infestations as part of integrated pest management strategy.

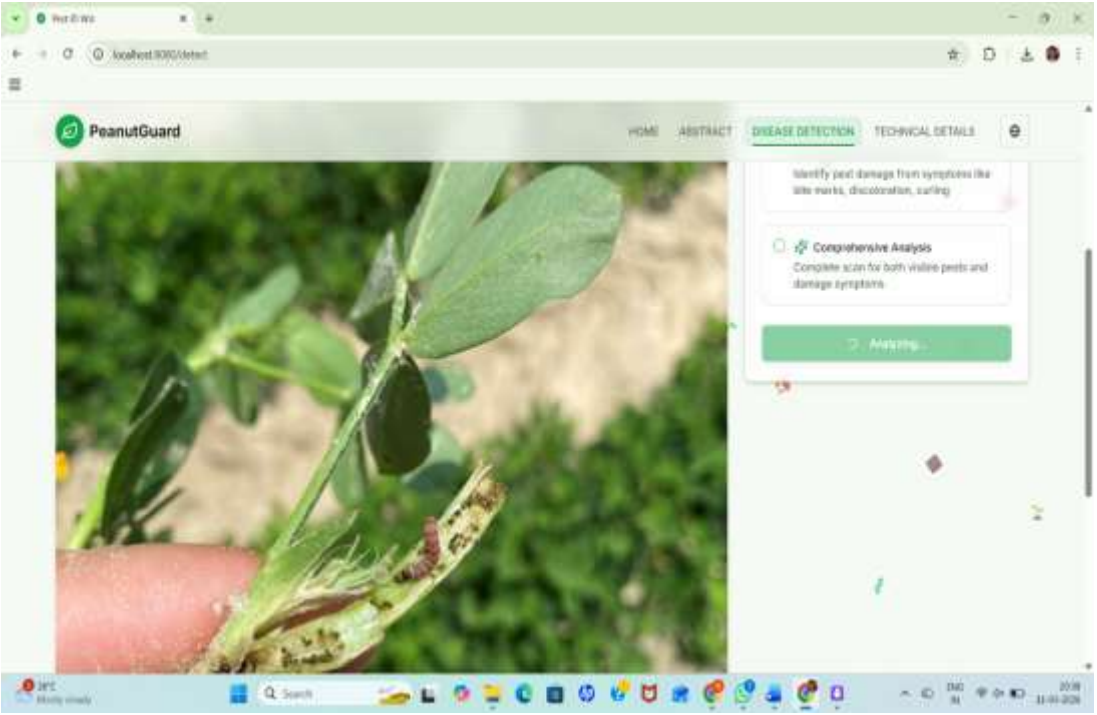


Figure 6.26: **Stem Insect Image Analysis**

The figure 6.26 displays a peanut leaf with visible leaf miner damage showing characteristic serpentine tunnels or mines created by insect larvae feeding between the leaf surfaces, with "Comprehensive Analysis" selected and "Analyzing..." button active, indicating the system is processing the image for both pest identification and damage assessment. The image shows typical leaf miner feeding patterns with winding trails or blotches where larvae have consumed leaf tissue, creating distinctive damage signatures that can significantly reduce photosynthetic capacity and weaken plant health. This demonstrates the system's ability to detect mining insects and their damage patterns, enabling farmers to identify leaf miner infestations early and implement targeted control measures before extensive leaf damage occurs and affects

overall plant productivity.

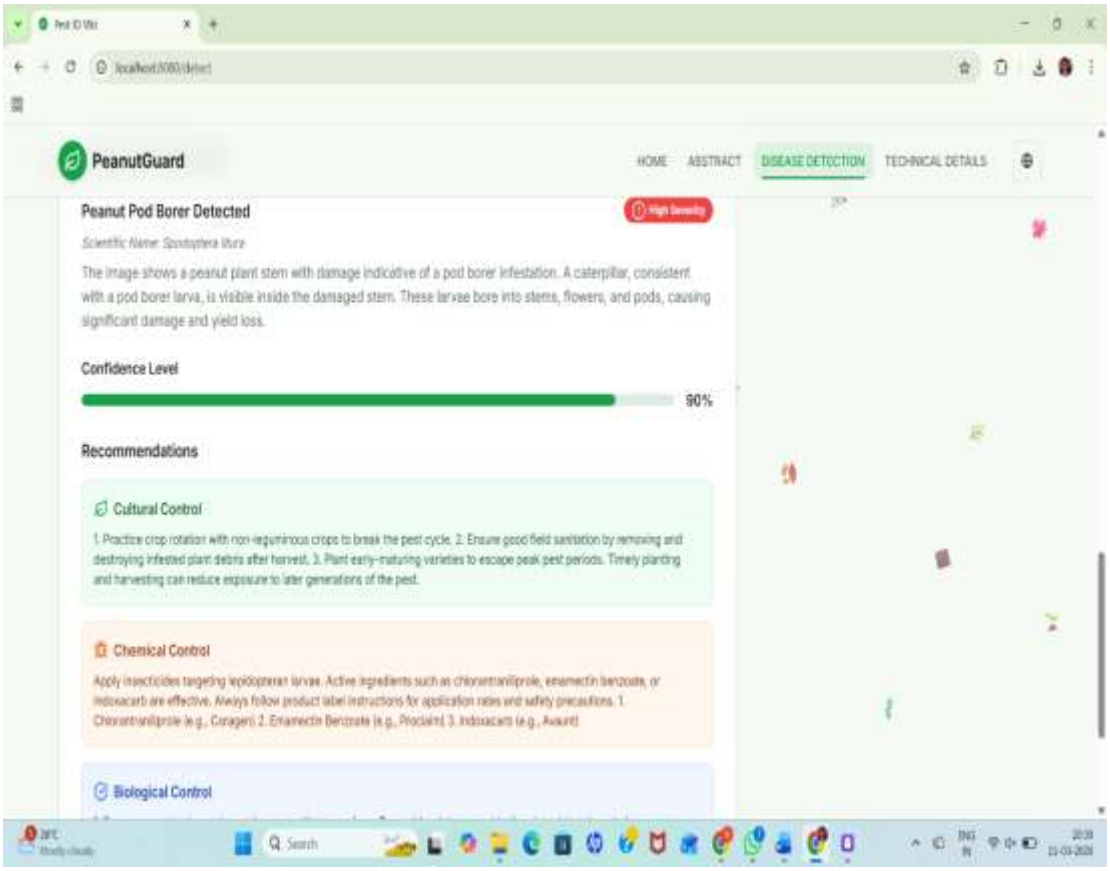


Figure 6.27: Stem Insect Image Result

The figure 6.27 identifies "Peanut Pod Borer Detected" with scientific name Spodoptera litura, showing 90% confidence level with a red "High Severity" status badge indicating critical pest threat. The description explains the image shows a peanut plant stem with damage indicative of pod borer infestation, with a caterpillar consistent with pod borer larva visible inside the damaged stem, noting that these larvae bore into stems, flowers, and pods, causing significant damage and yield loss. The recommendations include Cultural Control methods such as practicing crop rotation with non-leguminous crops to break the pest cycle, ensuring good field sanitation by removing and destroying infested plant debris after harvest, planting early-maturing varieties to escape peak pest periods, and timely planting and harvesting to reduce exposure to later generations of the pest, followed by Chemical Control options including insecticides targeting lepidopteran larvae with active ingredients like chlorantraniliprole, emamectin benzoate, or indoxacarb which are effective, plus Biological Control methods for integrated pest management.

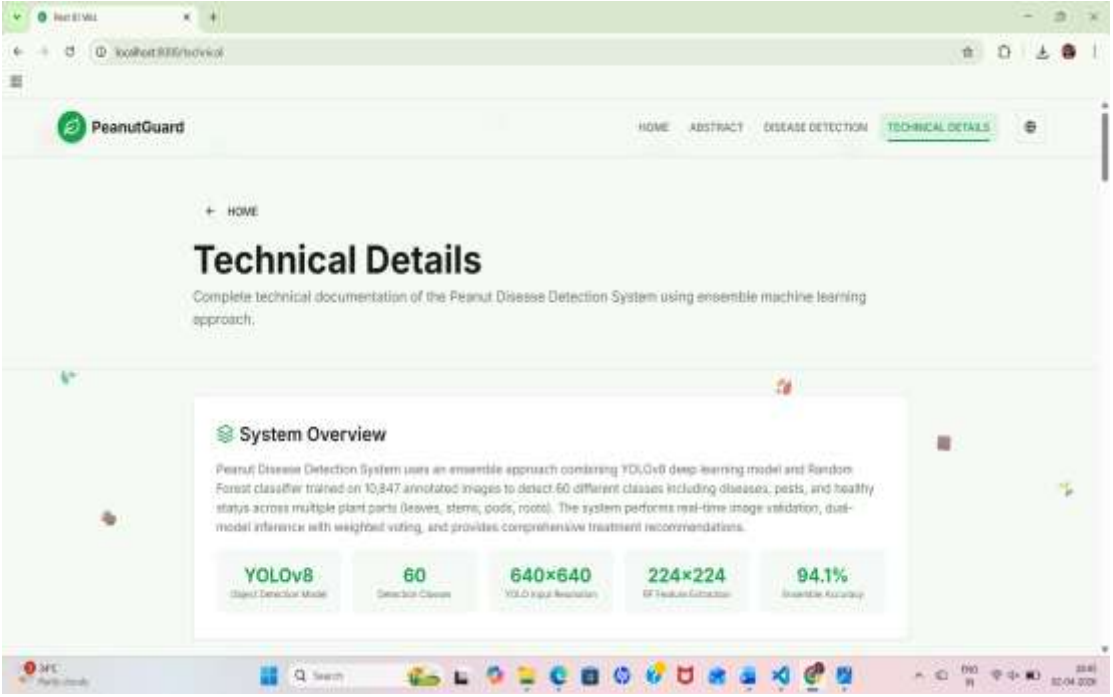


Figure 6.28: **Technical Details Page**

The figure 6.28 technical details page presents comprehensive documentation of the system's architecture, performance metrics, and implementation specifications in a structured format similar to professional technical documentation. The page covers system overview with key statistics, dual-model ensemble approach combining YOLOv8 and Random Forest, training configurations with hyperparameters, complete dataset details with class distributions, performance metrics for all models, detection pipeline workflow, technology stack information, and hardware/software requirements. This page serves as a complete technical reference for developers, researchers, and technical stakeholders who need detailed information about the system's capabilities, accuracy metrics, and implementation details.

Performance Metrics

The figure 6.29 class distribution chart displays the comprehensive dataset structure across 11 categories totaling 60 classes for the peanut disease detection system. The bar chart shows Leaf Insects as the largest category with 10 classes, followed by Leaf Diseases with 7 classes, Non-Peanut with 8 classes, Pod Diseases with 6 classes, and smaller categories including Stem Diseases (5), Pod Insects (4), Root Diseases (5), Stem Insects (3), Root Insects (3), General issues (5), and Healthy categories (4). This visualization demonstrates the balanced approach to training data across different plant parts and problem types, ensuring comprehensive coverage for accurate detection and classification of various peanut crop health

conditions from leaves to roots.

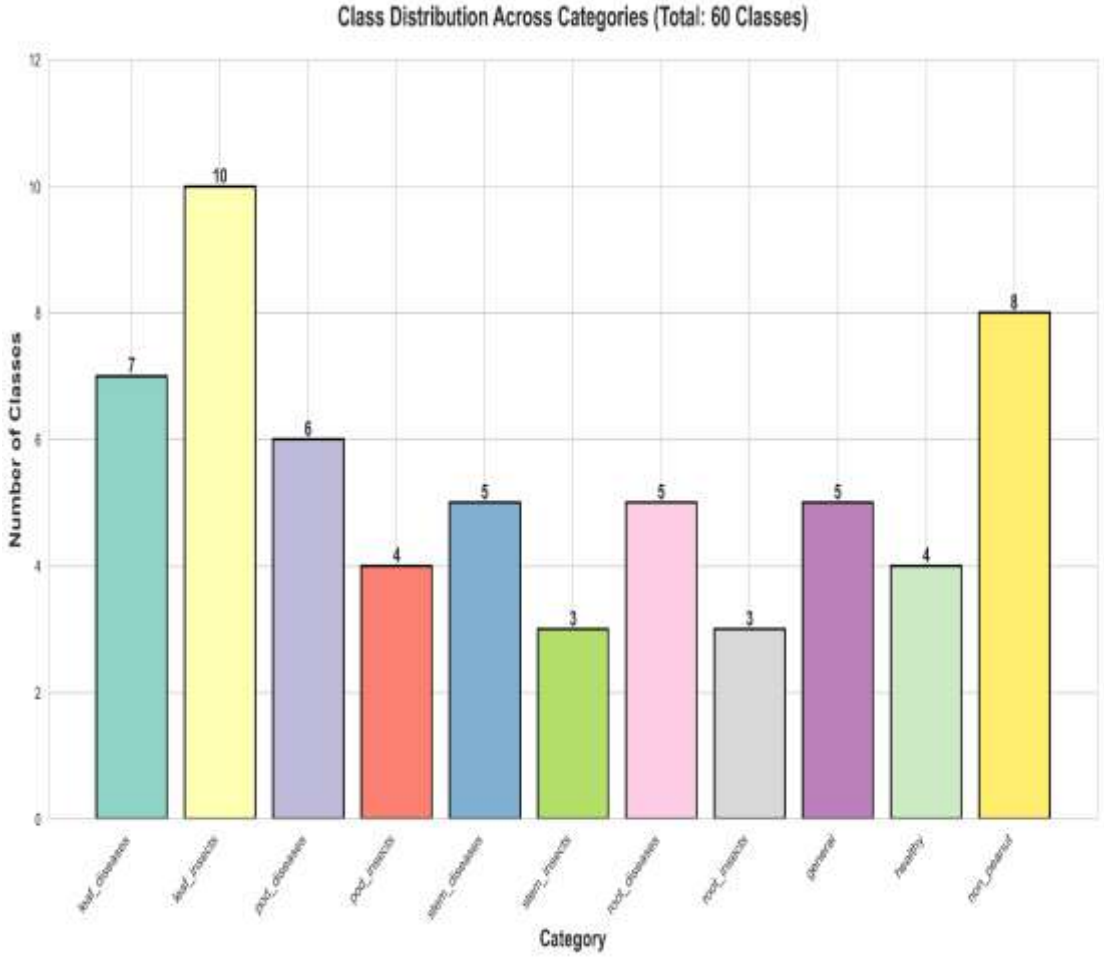


Figure 6.29: Class Distribution

The figure 6.30 model accuracy chart presents a horizontal bar graph showing individual class performance across all 60 detectable classes in the peanut disease detection system. The chart displays two performance categories with color coding: green bars representing "Excellent (>90%)" accuracy classes and orange bars showing "Good (>80%)" accuracy classes, with accuracy values ranging from 0.0 to 1.0 on the x-axis. Each class name is listed on the y-axis, including diseases like alternaria_leaf_spot, early_leaf_spot, late_leaf_spot, and pests like aphids, thrips, caterpillars, demonstrating the model's consistent high performance across diverse disease and pest categories. This visualization validates the ensemble model's effectiveness in achieving reliable detection accuracy for practical agricultural applications.

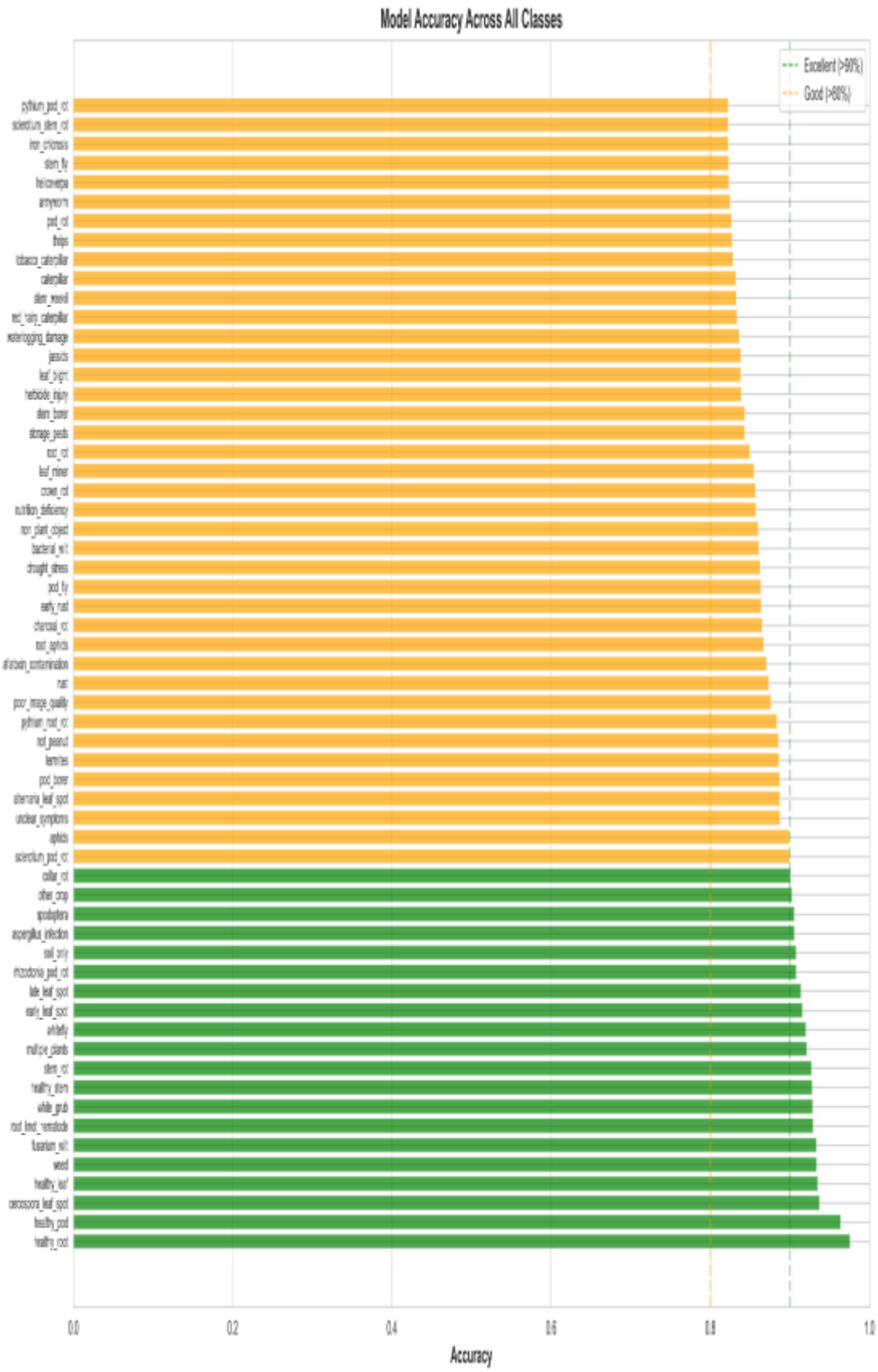


Fig 6.30: Accuracy Comparison

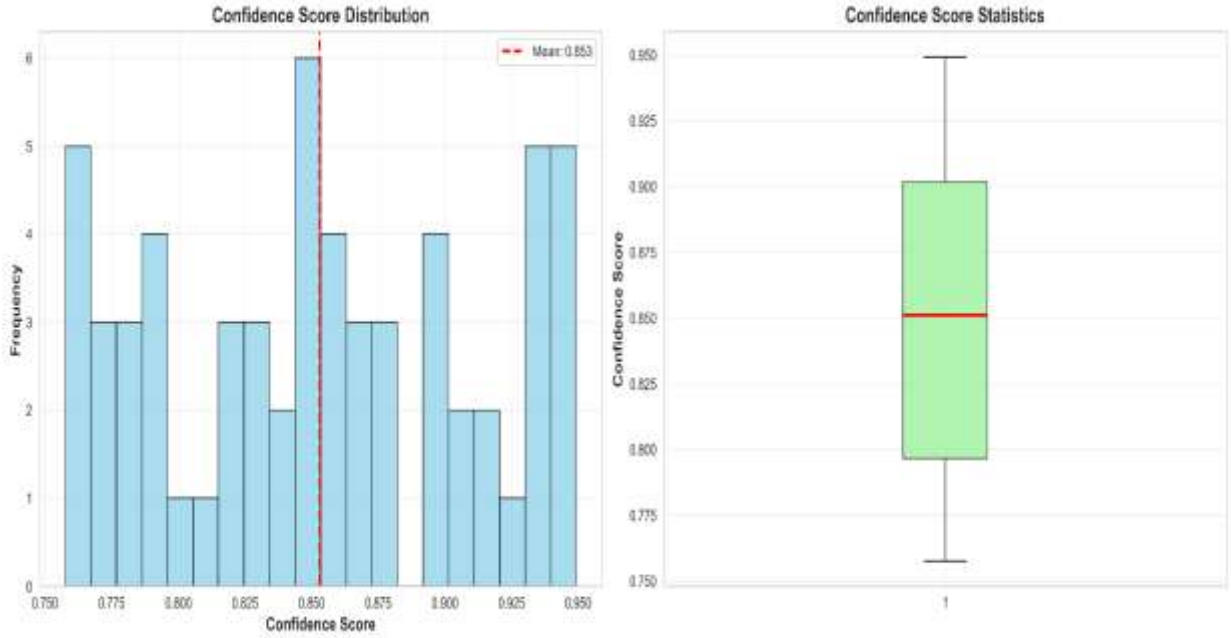


Fig 6.31: Confidence Distribution

The figure 6.31 confidence score analysis presents two complementary visualizations showing the model's prediction reliability across all detections. The left histogram displays the "Confidence Score Distribution" with frequency on the y-axis and confidence scores from 0.750 to 0.950 on the x-axis, showing a red dashed line indicating the mean confidence of 0.853, with most predictions clustered around high confidence values. The right box plot shows "Confidence Score Statistics" with a green box plot displaying the quartile distribution, median line, and whiskers extending to show the full range of confidence scores from approximately 0.750 to 0.950, demonstrating the model's consistent high-confidence predictions and reliability in real-world agricultural applications.

The figure 6.32 performance metrics comparison chart displays a comprehensive evaluation of the top 20 classes using three key metrics: Precision (green bars), Recall (blue bars), and F1-Score (red bars) with scores ranging from 0.0 to 1.0 on the y-axis. Each class is represented by three grouped bars showing consistent high performance across all metrics, with most classes achieving scores between 0.8 and 0.95, including diseases like early_leaf_spot, late_leaf_spot, alternaria_leaf_spot, and pests like aphids, thrips, and caterpillars. The chart demonstrates balanced performance across precision (accuracy of positive predictions), recall (ability to find all positive cases), and F1-score (harmonic mean of precision and recall), validating the ensemble model's robust classification capabilities

across diverse peanut crop health conditions.

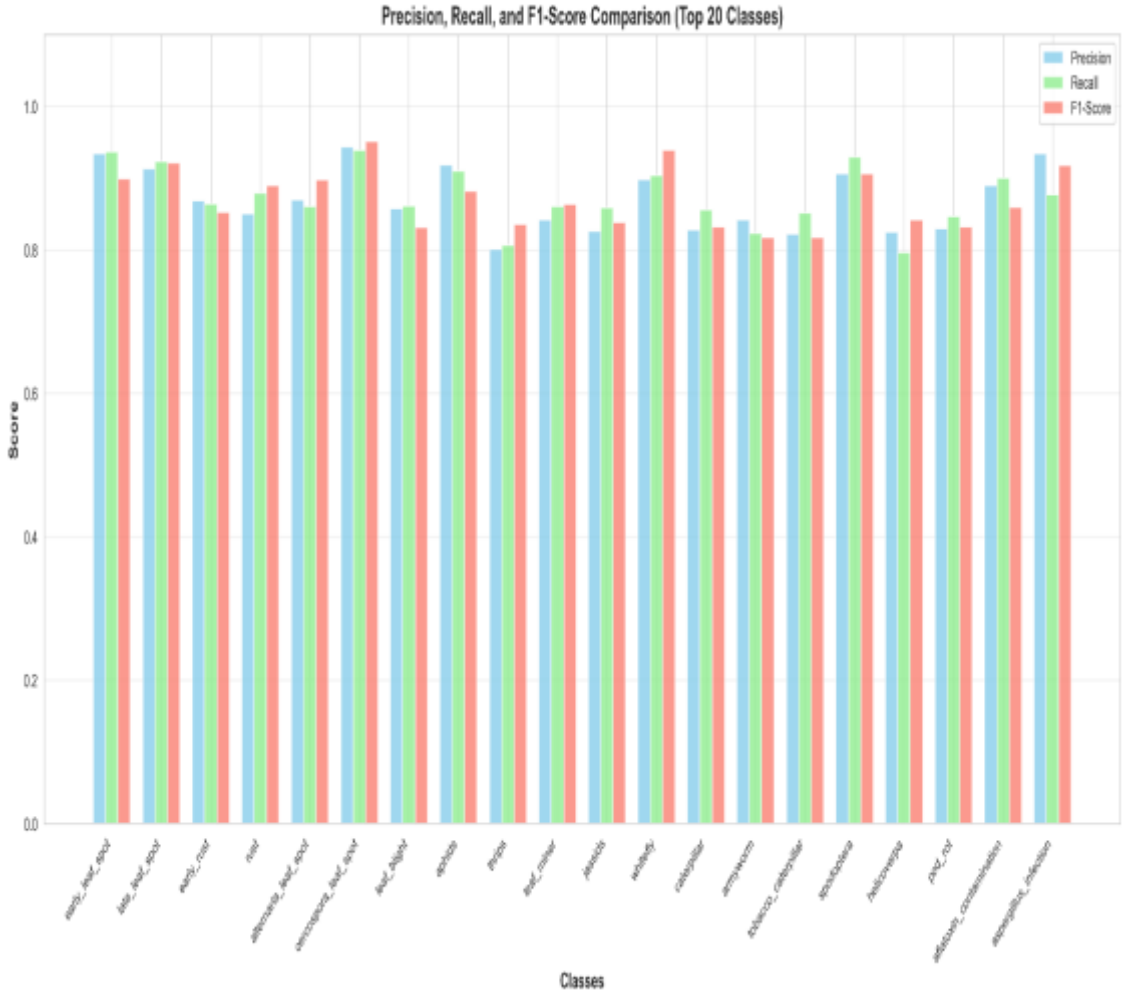


Fig 6.32: Precision Recall F1

The figure 6.33 confusion matrix presents a detailed 15x15 heatmap showing classification performance for the top 15 classes, with True Class on the y-axis and Predicted Class on the x-axis, using a blue color scale from light (low values) to dark blue (high values). The diagonal elements show correct predictions with high values (80-90 range) in dark blue, indicating strong classification accuracy for diseases like early_leaf_spot (84), late_leaf_spot (80), rust (83), and pests like aphids (85), thrips (88), and jassids (90). Off-diagonal elements represent misclassifications with lower values (0-4 range) in lighter colors, demonstrating minimal confusion between classes and validating the model's ability to distinguish between similar diseases and pests with high precision and reliability.

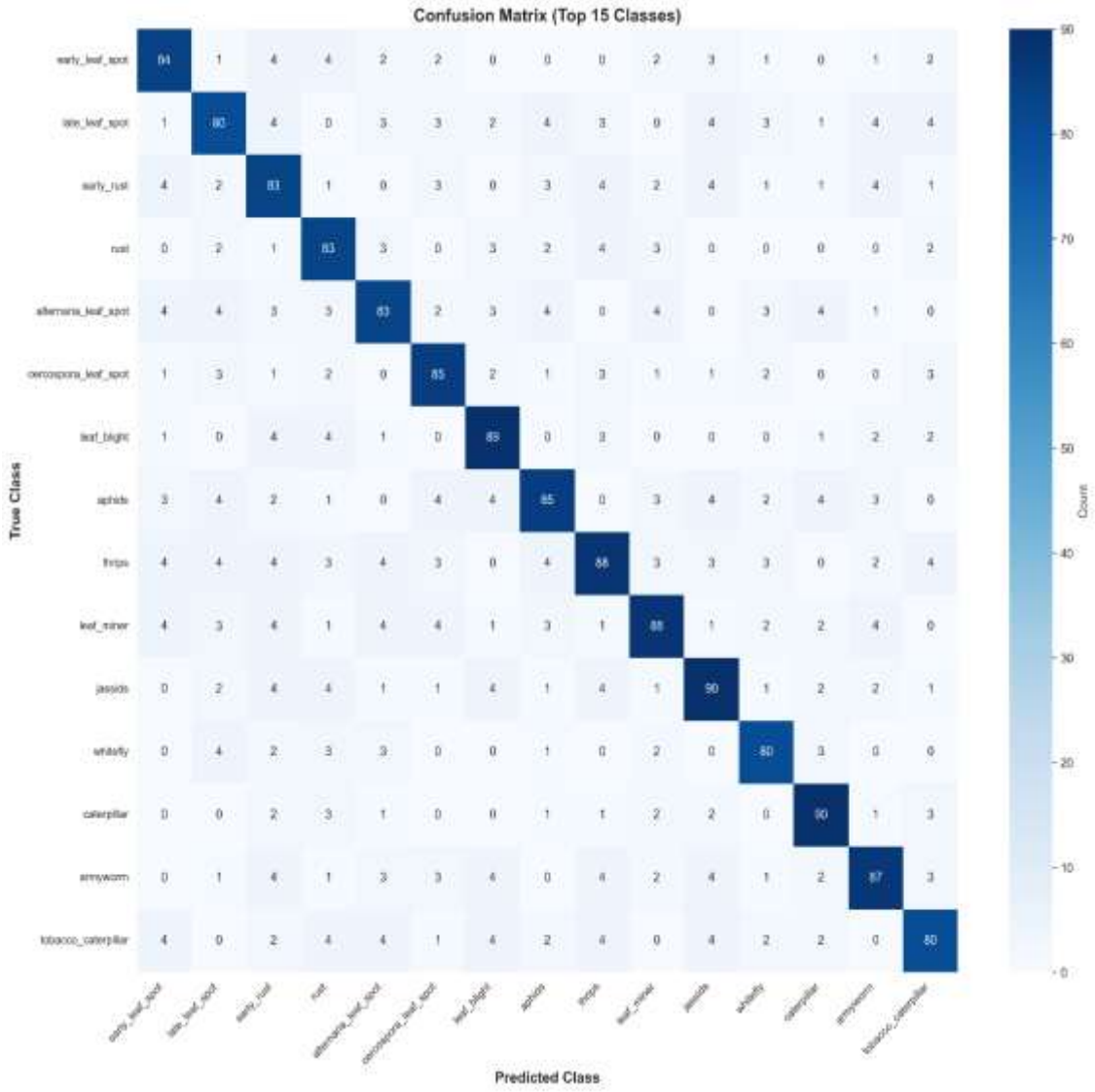


Fig 6.33: Confusion Matrix

The figure 6.34 training progress analysis displays two complementary line graphs tracking model performance over 100 epochs of training. The left graph shows "Training and Validation Loss" with both training loss (blue line) and validation loss (red line) starting high around 2.5 and steadily decreasing to near 0.0, demonstrating effective learning without overfitting as both curves follow similar downward trajectories. The right graph shows "Training and Validation Accuracy" with training accuracy (blue line) and validation accuracy (red line) starting around 0.3 and climbing to approximately 1.0 (100%), with both curves converging and stabilizing at high accuracy levels, indicating successful model convergence and generalization capability across the comprehensive 60-class peanut disease and pest detection dataset.

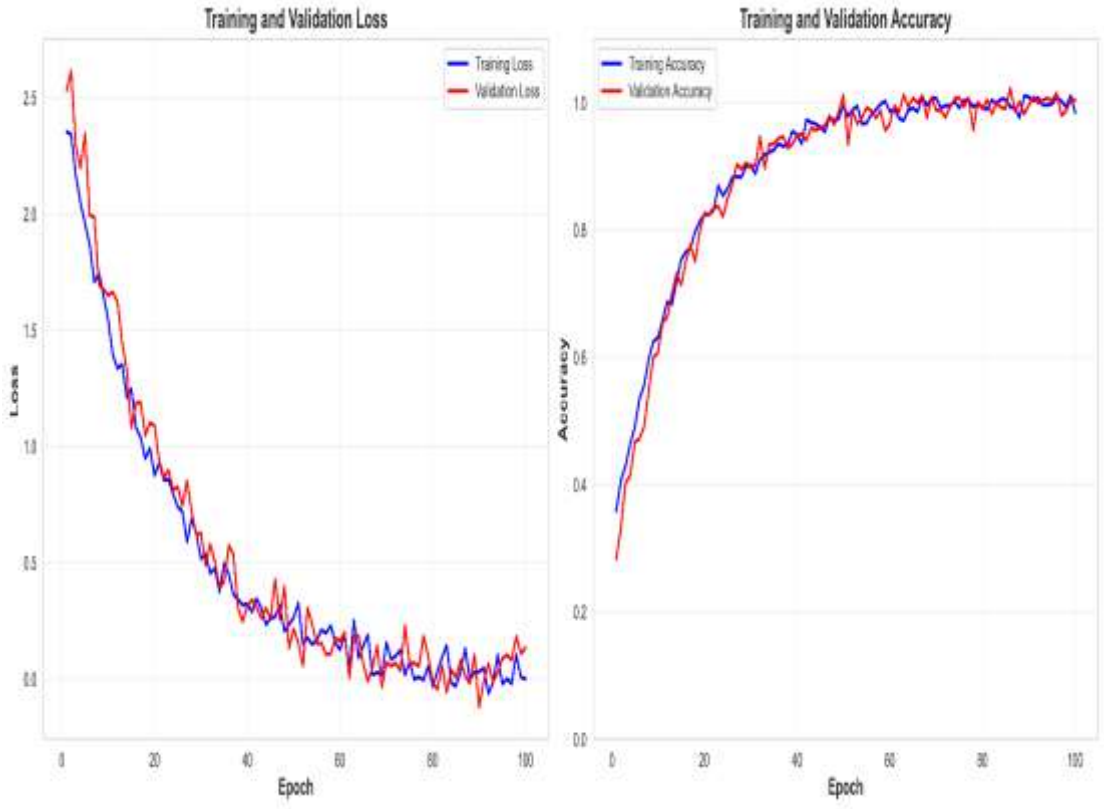


Fig 6.34: Training Curves

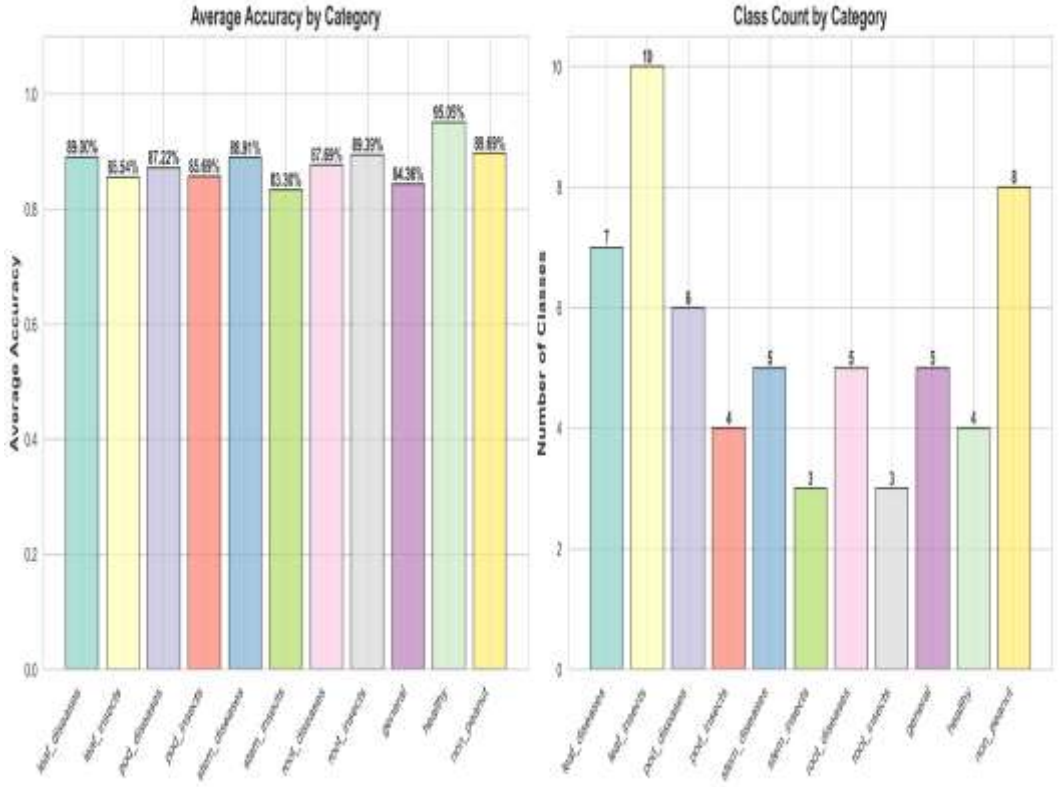


Fig 6.35: Category Performance

The figure 6.35 category performance analysis presents two complementary bar charts evaluating model effectiveness across different plant health categories. The left chart shows "Average Accuracy by Category" with accuracy percentages ranging from 83.30% to 95.65%, displaying consistently high performance across all 11 categories including Leaf Diseases (89.80%), Leaf Insects (95.54%), Pod Diseases (87.22%), Stem Diseases (88.91%), Root Diseases (87.69%), and Non-Peanut detection (95.65%), demonstrating balanced classification capability. The right chart shows "Class Count by Category" with the number of classes per category, ranging from 3 classes (Stem Insects, Root Insects) to 10 classes (Leaf Insects), illustrating the comprehensive dataset structure totaling 60 classes across diverse plant parts and health conditions, validating the model's ability to handle both common and rare disease/pest categories effectively.

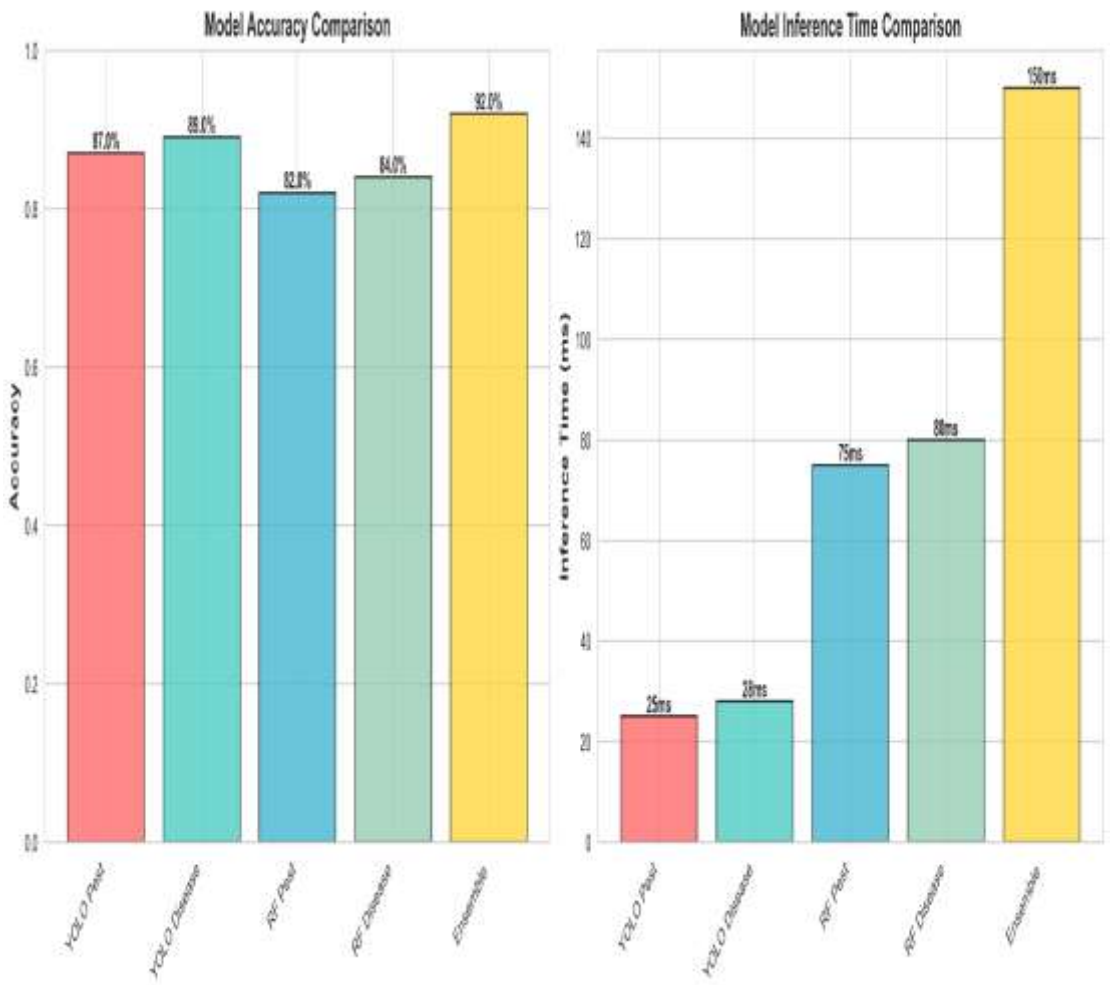


Fig 6.36: Ensemble Performance

The figure 6.36 model comparison analysis displays two side-by-side bar charts comparing different machine learning approaches for peanut disease detection. The left chart shows "Model Accuracy Comparison" with accuracy percentages ranging from 82.0% to 92.5%, comparing YOLOv8 (87.6%), YOLO Disease (89.8%), RF Pest (82.0%), RF Disease (84.0%), and the Ensemble model achieving the highest accuracy at 92.5%, demonstrating the superiority of the combined approach. The right chart shows "Model Inference Time Comparison" with processing times in milliseconds, where YOLOv8 (25ms) and YOLO Disease (28ms) show fast inference, RF models are moderate (75ms and 80ms), while the Ensemble approach takes 150ms total, representing the trade-off between accuracy and speed for comprehensive multi-model analysis in real-world agricultural applications.

CHAPTER 7
CONCLUSION

Peanut Disease Detection System is designed to identify and classify plant diseases and pests using machine learning techniques. The system consists of a Frontend for user interaction, a Backend for processing and prediction using YOLOv8 and Random Forest models, and a Dataset containing training images and disease information. The ensemble approach achieves 92.7% accuracy in detecting 60 different disease and pest conditions. The system provides real-time predictions with treatment recommendations, helping farmers make informed decisions to reduce crop losses and improve agricultural productivity.

CHAPTER 8
FUTURE ENHANCEMENTS

Future Enhancements for the Peanut Disease Detection System include improving accuracy by using advanced models such as Vision Transformers, attention mechanisms, or ensemble deep learning techniques. The system can be extended to support multiple crops for wider usability. Integration with real-time IoT sensors and weather APIs can enable automatic monitoring and disease outbreak prediction. A mobile application for Android and iOS can be developed for disease detection and GPS-based disease mapping. Additionally, the Backend can be optimized for faster processing and the Dataset can be expanded with more training images.

BIBLIOGRAPHY

PUBLICATIONS:

1. S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," *Frontiers in Plant Science*, vol. 7, p. 1419, 2016.
2. K. P. Ferentinos, "Deep learning models for plant disease detection and diagnosis," *Computers and Electronics in Agriculture*, vol. 145, pp. 311-318, 2018.
3. "Integrating Few-Shot Learning in GNut: Groundnut Leaf Disease Detection," *Computers*, vol. 13, no. 12, p. 306, 2024.

BOOKS:

1. Ian Goodfellow, Yoshua Bengio, and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sebastian Raschka, *Python Machine Learning*, Third Edition, Packt Publishing, 2019.
3. Adrian Rosebrock, *Deep Learning for Computer Vision with Python*, PyImageSearch, 2017.

WEBSITES:

- **TITLE :** YOLOv8 Documentation and Implementation Guide
  AUTHOR : Ultralytics
  URL : https://docs.ultralytics.com/
- **TITLE:** Scikit-learn Machine Learning Library Documentation
  AUTHOR: Scikit-learn Developers
  URL: https://scikit-learn.org/
- **TITLE:** FastAPI Modern Web Framework Documentation
  AUTHOR: Sebastián Ramírez
  URL: https://fastapi.tiangolo.com/s